

DATA ANALYTICS MAIN PROJECT USING PYTHON

Submitted by Sakthiraj R Nair

Project Title:

**Black Friday Sales Analysis
Using Python**

Project Scope:

- 1. Data Loading and Initial Overview
- 2. Data Pre-processing
- 3. Exploratory Data Analysis (EDA)
- 4. Visualizations
- 5. Insight Generation and Report

Project Domain : Sales and E-commerce

Data Source : Kaggle

What is Black Friday Sales?

Black Friday is a major shopping event that takes place every year, mainly in the United States, on the Friday after Thanksgiving. It is known for large discounts, special offers, and increased customer purchases.

What is a Black Friday Sales Dataset?

A Black Friday sales dataset contains information about customers and the products they purchased during a Black Friday shopping event.

Project Objective

The objective of this project is to analyze Black Friday sales data using Python.

The analysis focuses on customer purchasing behavior, product categories, demographic patterns, and sales performance.

I. Data Loading and Initial Overview

Step 1: Importing Libraries

CODE

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

import warnings
warnings.filterwarnings('ignore')

print("Libraries imported successfully")
```

OUTPUT / RESULT

```
Libraries imported successfully
```

The message confirms that pandas, NumPy, Matplotlib, and Seaborn were imported successfully.

Warnings were also configured to be hidden, so routine warning messages will not clutter the notebook.

Step 2: loading Dataset

CODE

```
df = pd.read_csv("Black-Friday-OriginalDataSet.csv")
```

OUTPUT / RESULT

No printed/table output is stored for this cell. A successful run without an exception is the expected result.

If the cell runs without an error, the CSV has been read successfully into the DataFrame named df.

Step 3: Display First 5 Records

CODE
df.head()

OUTPUT / RESULT

	User_ID	Product_ID	Gender	Age	Occupation	City_Category	\
0	1000001	P00069042	F	0-17	10	A	
1	1000001	P00248942	F	0-17	10	A	
2	1000001	P00087842	F	0-17	10	A	
3	1000001	P00085442	F	0-17	10	A	
4	1000002	P00285442	M	55+	16	C	
	Stay_In_Current_City_Years		Marital_Status		Product_Category_1		\
0	2		0		3		
1	2		0		1		
2	2		0		12		
3	2		0		12		
4	4+		0		8		
	Product_Category_2		Product_Category_3		Purchase		
0	NaN		NaN		8370		
1	6.0		14.0		15200		
2	NaN		NaN		1422		
3	14.0		NaN		1057		
4	NaN		NaN		7969		

INTERPRETATION

The first five records are displayed. This confirms the dataset structure and shows examples of customer, product, demographic, category, and Purchase fields. NaN values are visible in Product_Category_2 and Product_Category_3, indicating missing data.

Step 4: Display Last 5 Records

CODE
df.tail()

OUTPUT / RESULT

User_ID	Product_ID		Gender	Age	Occupation	City_Category	\	
550063	1006033	P00372445	M	51-55	13	B		
550064	1006035	P00375436	F	26-35	1	C		
550065	1006036	P00375436	F	26-35	15	B		
550066	1006038	P00375436	F	55+	1	C		
550067	1006039	P00371644	F	46-50	0	B		
	Stay_In_Current_City_Years			Marital_Status		Product_Category_1		\
550063	1			1		20		
550064	3			0		20		
550065	4+			1		20		
550066	2			0		20		
550067	4+			1		20		
	Product_Category_2		Product_Category_3		Purchase			
550063	NaN		NaN		368			
550064	NaN		NaN		371			
550065	NaN		NaN		137			
550066	NaN		NaN		365			
550067	NaN		NaN		490			

INTERPRETATION

The last five records are displayed. This checks the end of the loaded dataset and confirms that the DataFrame contains records through index 550067.

Step 5: Find Number of Rows and Columns

CODE

```
print("Number of Rows:", df.shape[0])  
print("Number of Columns:", df.shape[1])
```

OUTPUT / RESULT

```
Number of Rows: 550068  
Number of Columns: 12
```

INTERPRETATION

The dataset contains 550,068 rows and 12 columns. In other words, the original dataset has 550,068 transaction records and 12 variables.

Step 6: Display All Column Names

CODE

```
print("Columns in the dataset:")  
for column in df.columns:  
    print(column)
```

OUTPUT / RESULT

```
Columns in the dataset:  
User_ID  
Product_ID  
Gender  
Age  
Occupation  
City_Category  
Stay_In_Current_City_Years  
Marital_Status  
Product_Category_1  
Product_Category_2  
Product_Category_3  
Purchase
```

INTERPRETATION

The output lists all 12 column names: User_ID, Product_ID, Gender, Age, Occupation, City_Category, Stay_In_Current_City_Years, Marital_Status, Product_Category_1, Product_Category_2, Product_Category_3, and Purchase.

Step 7: Check Data Types

```
CODE
df.dtypes
```

OUTPUT / RESULT

```
User_ID          int64
Product_ID       object
Gender           object
Age             object
Occupation       int64
City_Category    object
Stay_In_Current_City_Years  object
Marital_Status   int64
Product_Category_1  int64
Product_Category_2  float64
Product_Category_3  float64
Purchase         int64
dtype: object
```

INTERPRETATION

The output shows the data type of every column. User_ID, Occupation, Marital_Status, Product_Category_1 and Purchase are int64; Product_Category_2 and Product_Category_3 are float64; the remaining fields are stored as object/text.

Step 8: Complete Dataset Information

```
CODE
df.info()
```

OUTPUT / RESULT

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 550068 entries, 0 to 550067
Data columns (total 12 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   User_ID              550068 non-null  int64
 1   Product_ID           550068 non-null  object
 2   Gender               550068 non-null  object
 3   Age                 550068 non-null  object
 4   Occupation           550068 non-null  int64
 5   City_Category        550068 non-null  object
 6   Stay_In_Current_City_Years  550068 non-null  object
 7   Marital_Status       550068 non-null  int64
 8   Product_Category_1   550068 non-null  int64
 9   Product_Category_2   376430 non-null  float64
10   Product_Category_3   166821 non-null  float64
11   Purchase             550068 non-null  int64
dtypes: float64(2), int64(5), object(5)
memory usage: 50.4+ MB
```

INTERPRETATION

The DataFrame has 550,068 rows and 12 columns. Product_Category_2 has 376,430 non-null values and Product_Category_3 has 166,821 non-null values, so these two columns contain missing values. The dataset uses about 50.4 MB of memory.

Step 9: Statistical Summary

CODE

```
df.describe()
```

OUTPUT / RESULT

	User_ID	Occupation	Marital_Status	Product_Category_1
count	5.500680e+05	550068.000000	550068.000000	550068.000000
mean	1.003029e+06	8.076707	0.409653	5.404270
std	1.727592e+03	6.522660	0.491770	3.936211
min	1.000001e+06	0.000000	0.000000	1.000000
25%	1.001516e+06	2.000000	0.000000	1.000000
50%	1.003077e+06	7.000000	0.000000	5.000000
75%	1.004478e+06	14.000000	1.000000	8.000000
max	1.006040e+06	20.000000	1.000000	20.000000

	Product_Category_2	Product_Category_3	Purchase
count	376430.000000	166821.000000	550068.000000
mean	9.842329	12.668243	9263.968713
std	5.086590	4.125338	5023.065394
min	2.000000	3.000000	12.000000
25%	5.000000	9.000000	5823.000000
50%	9.000000	14.000000	8047.000000
75%	15.000000	16.000000	12054.000000
max	18.000000	18.000000	23961.000000

INTERPRETATION

The numerical summary shows, among other values, an average Purchase of about 9,263.97, a median Purchase of 8,047, a minimum of 12, and a maximum of 23,961. Purchase therefore varies substantially across transactions.

Step 10: Categorical Summary

CODE

```
df.describe(include='object')
```

OUTPUT / RESULT

	Product_ID	Gender	Age	City_Category	Stay_In_Current_City_Years
count	550068	550068	550068	550068	550068
unique	3631	2	7	3	5
top	P00265242	M	26-35	B	1
freq	1880	414259	219587	231173	193821

INTERPRETATION

The categorical summary describes text columns such as Product_ID, Gender, Age, City_Category and Stay_In_Current_City_Years. It helps identify the number of unique categories and the most frequent category in each field.

II. Data Pre-processing

Step 11: Check Missing Values

CODE

```
df.isnull().sum()
```

OUTPUT / RESULT

```
User_ID      0
Product_ID   0
Gender        0
Age           0
Occupation    0
City_Category 0
Stay_In_Current_City_Years 0
Marital_Status 0
Product_Category_1  0
Product_Category_2 173638
Product_Category_3 383247
Purchase      0
dtype: int64
```

INTERPRETATION

The missing-value count identifies missing entries. Product_Category_2 has 173,638 missing values and Product_Category_3 has 383,247 missing values; the other columns have no missing values.

Step 12: Missing Values Percentage

CODE

```
missing_percentage = (df.isnull().sum() / len(df)) * 100
```

```
missing_percentage.sort_values(ascending=False)
```

OUTPUT / RESULT

```
Product_Category_3    69.672659
Product_Category_2    31.566643
User_ID                0.000000
Product_ID            0.000000
Gender                0.000000
Age                  0.000000
Occupation            0.000000
City_Category         0.000000
Stay_In_Current_City_Years 0.000000
Marital_Status        0.000000
Product_Category_1    0.000000
Purchase              0.000000
dtype: float64
```

INTERPRETATION

The missing-value percentages rank the columns by the proportion of missing data. Product_Category_3 has the largest missing proportion, followed by Product_Category_2; the other columns have 0% missing values.

Step 13: Visualize Missing Values

CODE

```
plt.figure(figsize=(10,5))

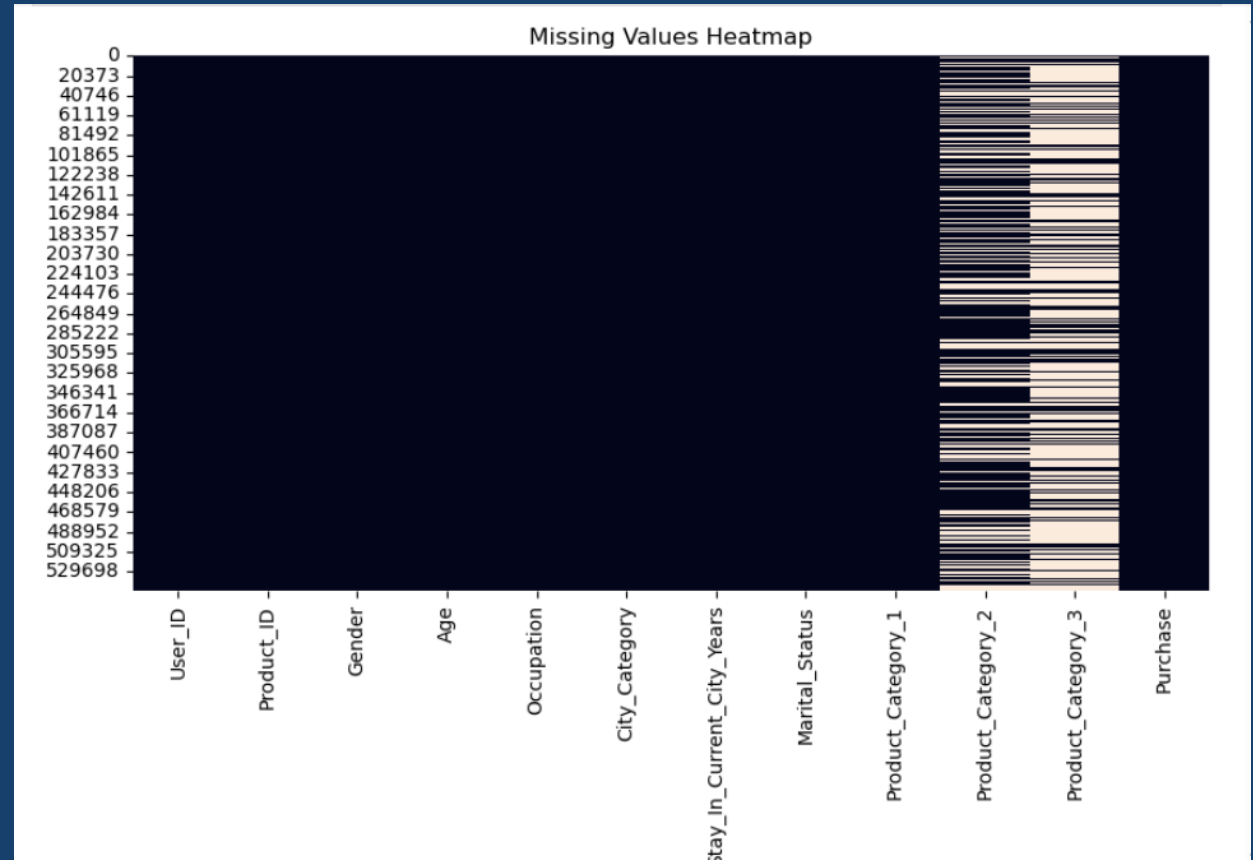
sns.heatmap(df.isnull(), cbar=False)

plt.title("Missing Values Heatmap")
plt.show()
```

INTERPRETATION

The heatmap visualizes where missing values occur across the dataset. It is used to see the concentration of missing data, especially in Product_Category_2 and Product_Category_3.

OUTPUT / RESULT



Step 14: Check Duplicate Records

CODE

```
duplicate_count = df.duplicated().sum()

print("Number of duplicate records:",
      duplicate_count)
```

OUTPUT/RESULT

```
Number of duplicate records: 0
```

INTERPRETATION

The output reports the number of completely duplicated rows.

A result of 0 means no exact duplicate records were found.

Step 15: Handle Missing Values

CODE

```
df['Product_Category_2'] = df['Product_Category_2'].fillna(0)

df['Product_Category_3'] = df['Product_Category_3'].fillna(0)
```

INTERPRETATION

Missing values in Product_Category_2 and Product_Category_3 are replaced with 0. This is the notebook's chosen cleaning approach; 0 acts as the replacement value for previously missing category entries.

Step 16: Verify null values Again

CODE

```
df.isnull().sum()
```

OUTPUT / RESULT

```
User_ID      0
Product_ID    0
Gender        0
Age           0
Occupation    0
City_Category 0
Stay_In_Current_City_Years 0
Marital_Status 0
Product_Category_1 0
Product_Category_2 0
Product_Category_3 0
Purchase      0
dtype: int64
```

INTERPRETATION

After the replacement, the missing-value check shows no missing values in the DataFrame. The cleaning step therefore removed the NaN values from the two product-category columns.

Step 17: Check Data Types Again

CODE

```
df.dtypes
```

OUTPUT / RESULT

```
User_ID      int64
Product_ID    object
Gender        object
Age           object
Occupation    int64
City_Category object
Stay_In_Current_City_Years object
Marital_Status int64
Product_Category_1 int64
Product_Category_2 float64
Product_Category_3 float64
Purchase      int64
dtype: object
```

INTERPRETATION

The data types remain consistent after the missing-value replacement. Product_Category_2 and Product_Category_3 remain numeric float columns because they now contain zeros as well as category values.

Step 18: Check Unique Values

CODE

```
for column in df.columns:  
    print(column, ":", df[column].nunique())
```

OUTPUT / RESULT

```
User_ID : 5891  
Product_ID : 3631  
Gender : 2  
Age : 7  
Occupation : 21  
City_Category : 3  
Stay_In_Current_City_Years : 5  
Marital_Status : 2  
Product_Category_1 : 20  
Product_Category_2 : 18  
Product_Category_3 : 16  
Purchase : 18105
```

INTERPRETATION

The code counts distinct values in every column. This helps understand the number of customers, products, demographic categories, occupations, city categories, product categories and other unique values available.

Step 19: Unique Customers Analysis

CODE

```
unique_customers = df['User_ID'].nunique()  
  
print("Number of Unique Customers:",  
      unique_customers)
```

OUTPUT / RESULT

```
Number of Unique Customers: 5891
```

INTERPRETATION

There are 5,891 unique customers identified by User_ID. This means the 550,068 transaction rows come from 5,891 distinct customer IDs.

Step 20: Unique Products Analysis

CODE

```
unique_products =  
df['Product_ID'].nunique()  
  
print("Number of Unique  
Products:",  
unique_products)
```

OUTPUT / RESULT

```
Number of Unique Products: 3631
```

INTERPRETATION

There are 3,631 unique products identified by Product_ID. The dataset therefore contains many transactions across 3,631 distinct products.

Step 21: Create purchase_level (Derived column)

CODE

```
def purchase_level(value):  
    if value < 5000:  
        return 'Low'  
    elif value < 10000:  
        return 'Medium'  
    else:  
        return 'High'  
  
df['Purchase_Level'] =  
df['Purchase'].apply(purchase_level)  
  
df[['Purchase',  
    'Purchase_Level']].head()
```

OUTPUT / RESULT

	Purchase	Purchase_Level
0	8370	Medium
1	15200	High
2	1422	Low
3	1057	Low
4	7969	Medium

INTERPRETATION

A new Purchase_Level column is created using the rules: below 5,000 = Low; 5,000 to below 10,000 = Medium; 10,000 or more = High. The first five rows show the resulting labels alongside Purchase values.

Step 22: Overall Sales Analysis

CODE

```
total_sales = df['Purchase'].sum()
average_purchase = df['Purchase'].mean()
minimum_purchase = df['Purchase'].min()
maximum_purchase = df['Purchase'].max()

print("Total Sales:", total_sales)
print("Average Purchase:", average_purchase)
print("Minimum Purchase:", minimum_purchase)
print("Maximum Purchase:", maximum_purchase)
```

OUTPUT / RESULT

```
Total Sales: 5095812742
Average Purchase: 9263.968712959126
Minimum Purchase: 12
Maximum Purchase: 23961
```

INTERPRETATION

Total sales are 5,095,812,742. The average transaction is about 9,263.97, with a minimum of 12 and a maximum of 23,961. These four measures provide the overall sales baseline.

III.

Exploratory Data Analysis (EDA)

Step 23: Check Age Distribution

CODE

```
df['Age'].value_counts()
```

OUTPUT / RESULT

```
Age
26-35    219587
36-45    110013
18-25     99660
46-50     45701
51-55     38501
55+       21504
0-17      15102
Name: count, dtype: int64
```

INTERPRETATION

The 26-35 age group has the most transaction records (219,587), while 0-17 has the fewest (15,102). This describes transaction frequency by age group, not unique-customer count.

Step 24: Gender Analysis

CODE

```
gender_count =  
df['Gender'].value  
_counts()
```

```
gender_count
```

OUTPUT / RESULT

```
Gender  
M      414259  
F      135809  
Name: count, dtype: int64
```

INTERPRETATION

There are 414,259 male transaction records and 135,809 female transaction records. The dataset therefore contains more male than female transaction rows.

Step 25: Age Group - Purchase Analysis

CODE

```
age_purchase =  
df.groupby('Age')['Purchase']  
.sum().sort_values(ascending=  
False)
```

```
age_purchase
```

OUTPUT / RESULT

```
Age  
26-35      2031770578  
36-45      1026569884  
18-25       913848675  
46-50       420843403  
51-55       367099644  
55+         200767375  
0-17        134913183  
Name: Purchase, dtype:  
int64
```

INTERPRETATION

The 26-35 age group contributes the largest total Purchase amount, 2,031,770,578. The 0-17 group contributes the smallest total, 134,913,183.

Step 26: Average Purchase by Age

CODE

```
age_avg_purchase =  
df.groupby('Age')['Purchase'].mean()  
  
age_avg_purchase
```

OUTPUT / RESULT

```
Age  
0-17      8933.464640  
18-25     9169.663606  
26-35     9252.690633  
36-45     9331.350695  
46-50     9208.625697  
51-55     9534.808031  
55+       9336.280459  
Name: Purchase, dtype: float64
```

INTERPRETATION

Average purchase differs by age group. The highest average is for 51-55 at about 9,534.81, while 0-17 has the lowest at about 8,933.46.

Step 27: Total Purchase by Gender

CODE

```
gender_purchase =  
df.groupby('Gender')['Purchase'].  
sum()  
  
gender_purchase
```

OUTPUT / RESULT

```
Gender  
F      1186232642  
M      3909580100  
Name: Purchase, dtype: int64
```

INTERPRETATION

Male transaction records account for 3,909,580,100 in total Purchase, compared with 1,186,232,642 for female records. This is total transaction value, not per-customer spending.

Step 28: Average Purchase by Gender

CODE

```
df.groupby('Gender')['Purchase'].mean()
```

OUTPUT / RESULT

```
Gender
F      8734.565765
M      9437.526040
Name: Purchase, dtype: float64
```

INTERPRETATION

Average Purchase is about 9,437.53 for male transactions and 8,734.57 for female transactions. This compares the average transaction amount by gender.

Step 29: City Category Analysis

CODE

```
df['City_Category'].value_counts()
```

OUTPUT / RESULT

```
City_Category
B      231173
C      171175
A      147720
Name: count, dtype: int64
```

INTERPRETATION

City category B has the most transaction records (231,173), followed by C (171,175) and A (147,720).

Step 30: City-wise Sales

CODE

```
city_sales =  
df.groupby('City_Category')['Purchase'].sum()  
  
city_sales
```

OUTPUT / RESULT

```
City_Category  
A      1316471661  
B      2115533605  
C      1663807476  
Name: Purchase, dtype: int64
```

INTERPRETATION

City category B has the largest total Purchase, 2,115,533,605. Category A has 1,316,471,661 and category C has 1,663,807,476.

Step 31: Average Purchase by City

CODE

```
df.groupby('City_Category')['Purchase'].mean()
```

OUTPUT / RESULT

```
City_Category  
A      8911.939216  
B     9151.300563  
C     9719.920993  
Name: Purchase, dtype: float64
```

INTERPRETATION

Average Purchase is highest in city category C at about 9,719.92 and lowest in A at about 8,911.94. This is an average transaction comparison, not a total-sales comparison.

Step 32: Marital Status Analysis

CODE

```
marital_sales = df.groupby('Marital_Status')['Purchase'].sum()

marital_sales
```

OUTPUT / RESULT

```
Marital_Status
0           3008927447
1           2086885295
Name: Purchase, dtype: int64
```

INTERPRETATION

Marital status 0 has total Purchase of 3,008,927,447, while status 1 has 2,086,885,295. The code uses the dataset's numeric status values without assigning labels to them.

Step 33: Product Category 1 Analysis

CODE

```
category1_sales =
df.groupby('Product_Category_1')['Purchase'].sum()

category1_sales =
category1_sales.sort_values(ascending=False)

category1_sales.head(10)
```

OUTPUT / RESULT

```
Product_Category_1
1           1910013754
5           941835229
8           854318799
6           324150302
2           268516186
3           204084713
16          145120612
11          113791115
10          100837301
15           92969042
Name: Purchase, dtype: int64
```

INTERPRETATION

The top 10 Product_Category_1 values by total Purchase are listed. Category 1 has the largest total at 1,910,013,754, followed by category 5 and category 8.

Step 34: Purchase by Product Category

CODE

```
category1_purchase =  
df.groupby('Product_Category_1')['Purchase'].sum()  
  
category1_purchase.sort_values(ascending=False).head(10)
```

OUTPUT / RESULT

```
Product_Category_1  
1      1910013754  
5       941835229  
8       854318799  
6       324150302  
2       268516186  
3       204084713  
16      145120612  
11      113791115  
10      100837301  
15       92969042  
Name: Purchase, dtype: int64
```

INTERPRETATION

This repeats the same total-sales ranking as cell 33 using a second variable name. It confirms the same top 10 Product_Category_1 categories by total Purchase.

Step 35: Average Purchase by Product Category

CODE

```
avg_category_purchase =  
df.groupby('Product_Category_1')['Purchase']  
  .mean()  
  
avg_category_purchase.sort_values(ascending=False).head(15)
```

OUTPUT / RESULT

```
Product_Category_1  
10      19675.570927  
7       16365.689600  
6       15838.478550  
9       15537.375610  
15      14780.451828  
16      14766.037037  
1       13606.218596  
14      13141.625739  
2       11251.935384  
17      10170.759516  
3       10096.705734  
8       7498.958078  
5       6240.088178  
11      4685.268456  
18      2972.864320  
Name: Purchase, dtype: float64
```

INTERPRETATION

The ranking is based on average Purchase rather than total Purchase. Category 10 has the highest average transaction value, about 19,675.57, among the displayed categories.

Step 36: Occupation Analysis

CODE

```
occupation_count =  
df['Occupation'].value_counts()  
  
occupation_count
```

OUTPUT / RESULT

```
Occupation  
4      72308  
0      69638  
7      59133  
1      47426  
17     40043  
20     33562  
12     31179  
14     27309  
2      26588  
16     25371  
6      20355  
3      17650  
10     12930  
5      12177  
15     12165  
11     11586  
19      8461  
13      7728  
18      6622  
9       6291  
8       1546  
Name: count, dtype: int64
```

INTERPRETATION

Occupation 4 has the most transaction records (72,308), while occupation 8 has the fewest (1,546). These are transaction counts by occupation code.

Step 37: Total Purchase by Occupation

CODE

```
occupation_purchase =  
df.groupby('Occupation')['Purchase'].sum()  
  
occupation_purchase.sort_values(ascending=False)
```

OUTPUT / RESULT

```
Occupation  
4      666244484  
0      635406958  
7      557371587  
1      424614144  
17     393281453  
12     305449446  
20     296570442  
14     259454692  
16     238346955  
2      238028583  
6      188416784  
3      162002168  
15     118960211  
10     115844465  
5      113649759  
11     106751618  
19      73700617  
13      71919481  
18      60721461  
9       54340046  
8       14737388  
Name: Purchase, dtype: int64
```

INTERPRETATION

Occupation 4 has the highest total Purchase, 666,244,484, followed by occupation 0 and occupation 7. The result compares total transaction across occupation codes.

Step 38: Stay in Current City Analysis

CODE

```
df['Stay_In_Current_City_Years'].value_counts()
```

OUTPUT / RESULT

```
Stay_In_Current_City_Years
1      193821
2      101838
3       95285
4+      84726
0       74398
Name: count, dtype: int64
```

INTERPRETATION

Customers/transactions with 1 year in the current city are most frequent (193,821 records). The least frequent group is 0 years (74,398 records).

Step 39: Purchase by City Stay Duration

CODE

```
stay_sales =
df.groupby('Stay_In_Current_City_Years')['Purchase'].sum()
```

```
stay_sales
```

OUTPUT / RESULT

```
Stay_In_Current_City_Years
0      682979229
1     1792872533
2      949173931
3      884902659
4+     785884390
Name: Purchase, dtype: int64
```

INTERPRETATION

The largest total Purchase is for 1 year in the current city, 1,792,872,533. The smallest is for 0 years, 682,979,229.

Step 40: Top 10 Customers by Total Purchase

CODE

```
top_customers = (  
    df.groupby('User_ID')['Purchase']  
        .sum()  
        .sort_values(ascending=False)  
        .head(10)  
    )  
  
top_customers
```

OUTPUT / RESULT

```
User_ID  
1004277    10536909  
1001680     8699596  
1002909     7577756  
1001941     6817493  
1000424     6573609  
1004448     6566245  
1005831     6512433  
1001015     6511314  
1003391     6477160  
1001181     6387961  
Name: Purchase, dtype:  
int64
```

INTERPRETATION

The ten customer IDs with the highest summed Purchase are shown. Customer 1004277 has the highest total at 10,536,909 among the listed customers.

Step 41: Top 10 Products by Sales Amount

CODE

```
top_products = (  
    df.groupby('Product_ID')['Purchase']  
        .sum()  
        .sort_values(ascending=False)  
        .head(10)  
    )  
  
top_products
```

OUTPUT / RESULT

```
Product_ID  
P00025442    27995166  
P00110742    26722309  
P00255842    25168963  
P00059442    24338343  
P00184942    24334887  
P00112142    24216006  
P00110942    23639564  
P00237542    23425576  
P00057642    23102780  
P00010742    22164153  
Name: Purchase, dtype: int64
```

INTERPRETATION

The ten products with the highest summed Purchase are shown. Product P00025442 has the highest total at 27,995,166 among the listed products.

Step 42: Overall Purchase Statistics

CODE

```
print("Total Sales:",
df['Purchase'].sum())
print("Average Purchase:",
df['Purchase'].mean())
print("Minimum Purchase:",
df['Purchase'].min())
print("Maximum Purchase:",
df['Purchase'].max())
```

OUTPUT / RESULT

```
Total Sales:
5095812742
Average Purchase:
9263.968712959126
Minimum Purchase: 12
Maximum Purchase:
23961
```

INTERPRETATION

This is another calculation of the overall sales baseline: total sales 5,095,812,742, average Purchase about 9,263.97, minimum 12 and maximum 23,961.

Step 43: Pivot Table

CODE

```
pivot_table = pd.pivot_table(
    df,
    values='Purchase',
    index='Age',
    columns='Gender',
    aggfunc='sum'
)
```

pivot_table

OUTPUT / RESULT

Gender	F	M
Age		
0-17	42385978	92527205
18-25	205475842	708372833
26-35	442976233	1588794345
36-45	243438963	783130921
46-50	116706864	304136539
51-55	89465997	277633647
55+	45782765	154984610

INTERPRETATION

The pivot table breaks total Purchase into Age × Gender. For example, the 26-35 male group has 1,588,794,345 in total Purchase, while 26-35 female has 442,976,233.

Step 44: Correlation Analysis

CODE

```
numeric_columns =  
df.select_dtypes(include=np.number)  
  
correlation = numeric_columns.corr()  
  
correlation
```

OUTPUT / RESULT

User_ID	Occupation	Marital_Status	Product_Category_1	\
User_ID		1.000000	-0.023971	0.020443
Occupation		-0.023971	1.000000	0.024280
Marital_Status		0.020443	0.024280	1.000000
Product_Category_1		0.003825	-0.007618	0.019888
Product_Category_2		0.003896	0.006712	0.001099
Product_Category_3		0.003605	0.012269	-0.004629
Purchase		0.004716	0.020833	-0.000463

	Product_Category_2	Product_Category_3	Purchase
User_ID	0.003896	0.003605	0.004716
Occupation	0.006712	0.012269	0.020833
Marital_Status	0.001099	-0.004629	-0.000463
Product_Category_1	-0.067877	-0.385534	-0.343703
Product_Category_2	1.000000	0.094750	0.052288
Product_Category_3	0.094750	1.000000	0.288501
Purchase	0.052288	0.288501	1.000000

INTERPRETATION

The correlation matrix measures linear association among numeric variables. Purchase has a correlation of -0.3437 with Product_Category_1, 0.2885 with Product_Category_3, and 0.0523 with Product_Category_2. Correlation indicates association, not causation.

Step 45: Gender and Age Analysis

CODE

```
gender_age_sales = df.groupby(['Age',  
'Gender'])['Purchase'].sum().reset_index()  
  
gender_age_sales.head()
```

OUTPUT / RESULT

	Age	Gender	Purchase
0	0-17	F	42385978
1	0-17	M	92527205
2	18-25	F	205475842
3	18-25	M	708372833
4	26-35	F	442976233

INTERPRETATION

The grouped table combines Age and Gender and calculates total Purchase for each combination. The first rows show that 0-17 male transactions total 92,527,205 versus 42,385,978 for females in that age group.

Step 46: City and Gender Analysis

```
CODE
city_gender_sales = df.groupby(
    ['City_Category', 'Gender']
)['Purchase'].sum().reset_index()

city_gender_sales
```

OUTPUT / RESULT

	City_Category	Gender	Purchase
0	A	F	306329915
1	A	M	1010141746
2	B	F	493617008
3	B	M	1621916597
4	C	F	386285719
5	C	M	1277521757

INTERPRETATION

The grouped table combines City_Category and Gender. For example, male transactions total 1,621,916,597 in city category B, while female transactions total 493,617,008 there.

Step 47: Age and City Analysis

```
CODE
age_city = (
    df.groupby(['Age', 'City_Category'])['Purchase']
      .sum()
      .reset_index()
)

age_city
```

OUTPUT / RESULT

	Age	City_Category	Purchase
0	0-17	A	21916841
1	0-17	B	48465500
2	0-17	C	64530842
3	18-25	A	243236868
4	18-25	B	390594232
5	18-25	C	280017575
6	26-35	A	660202334
7	26-35	B	837919708
8	26-35	C	533648536
9	36-45	A	239295720
10	36-45	B	433517875
11	36-45	C	353756289
12	46-50	A	63507243
13	46-50	B	188713201
14	46-50	C	168622959
15	51-55	A	57992372
16	51-55	B	165717109
17	51-55	C	143390163
18	55+	A	30320283
19	55+	B	50605980
20	55+	C	119841112

INTERPRETATION

The table combines Age and City_Category to show total Purchase for every age/city combination. It can be used to compare demographic sales across city categories.

Step 48: Find Highest Purchase Transaction

CODE

```
df.loc[df['Purchase'].idxmax()]
```

OUTPUT / RESULT

```
User_ID          1001474
Product_ID      P00052842
Gender          M
Age            26-35
Occupation       4
City_Category   A
Stay_In_Current_City_Years  2
Marital_Status  1
Product_Category_1  10
Product_Category_2  15.0
Product_Category_3  0.0
Purchase        23961
Purchase_Level   High
Name: 87440, dtype: object
```

INTERPRETATION

The row with the maximum Purchase is identified. The maximum transaction is 23,961 for User_ID 1001474, Product_ID P00052842, age 26-35, gender M, city A.

Step 49: Find Lowest Purchase Transaction

CODE

```
df.loc[df['Purchase'].idxmin()]
```

OUTPUT / RESULT

```
User_ID          1000050
Product_ID      P00370293
Gender          F
Age            26-35
Occupation       2
City_Category   A
Stay_In_Current_City_Years  1
Marital_Status  1
Product_Category_1  19
Product_Category_2  0.0
Product_Category_3  0.0
Purchase         12
Purchase_Level   Low
Name: 545946, dtype: object
```

INTERPRETATION

The row with the minimum Purchase is identified. The minimum transaction is 12 for User_ID 1000050, Product_ID P00370293, age 26-35, gender F, city A.

Step 51: Top 10 Highest Purchase Transactions

```
CODE
df.nlargest(10, 'Purchase')
```

Output / Result

	User_ID	Product_ID	Gender	Age	Occupation	City_Category	Stay_In_Current_City_Years	Marital_Status	Product_Category_1	Product_Category_2	Product_Category_3	Purchase	Purchase_Level
87440	1001474	P00052842	M	26-35	4	A	2	1	10	15.0	0.0	23961	High
93016	1002272	P00052842	M	26-35	0	C	1	0	10	15.0	0.0	23961	High
370891	1003160	P00052842	M	26-35	17	C	3	0	10	15.0	0.0	23961	High
292083	1003045	P00052842	M	46-50	1	B	2	1	10	15.0	0.0	23960	High
321782	1001577	P00052842	M	55+	0	C	1	1	10	15.0	0.0	23960	High
349658	1005848	P00119342	M	51-55	20	A	0	1	10	13.0	0.0	23960	High
503697	1005596	P00117642	M	36-45	12	B	1	0	10	16.0	0.0	23960	High
298378	1003947	P00116142	M	26-35	0	C	3	0	10	13.0	16.0	23959	High
437804	1001387	P00086242	F	51-55	13	B	1	1	10	0.0	0.0	23959	High
7542	1001178	P00116142	M	55+	0	C	1	1	10	13.0	16.0	23958	High

INTERPRETATION

The ten individual transactions with the highest Purchase amounts are displayed. This identifies high-value transactions, which are different from customers ranked by their total across many transactions.

Step 52: Top 10 Customers by purchase

CODE

```
top_customers = (  
    df.groupby('User_ID')['Purchase']  
    .sum()  
    .sort_values(ascending=False)  
    .head(10)  
)
```

top_customers

OUTPUT / RESULT

```
User_ID  
1004277    10536909  
1001680     8699596  
1002909     7577756  
1001941     6817493  
1000424     6573609  
1004448     6566245  
1005831     6512433  
1001015     6511314  
1003391     6477160  
1001181     6387961  
Name: Purchase, dtype: int64
```

INTERPRETATION

The ten customers with the highest total Purchase are recalculated. Customer 1004277 remains the highest with 10,536,909 in total Purchase.

Step 53: Top 10 Products

CODE

```
top_products = (  
  
df.groupby('Product_ID')['Purchase'  
'']  
    .sum()  
    .sort_values(ascending=False)  
    .head(10)  
)
```

top_products

OUTPUT / RESULT

```
Product_ID  
P00025442    27995166  
P00110742    26722309  
P00255842    25168963  
P00059442    24338343  
P00184942    24334887  
P00112142    24216006  
P00110942    23639564  
P00237542    23425576  
P00057642    23102780  
P00010742    22164153  
Name: Purchase, dtype: int64
```

INTERPRETATION

The ten products with the highest total Purchase are recalculated. Product P00025442 remains the highest with 27,995,166.

Step 54: Customer Purchase Summary

CODE

```
customer_summary = df.groupby('User_ID').agg(  
    Total_Purchase=('Purchase', 'sum'),  
    Average_Purchase=('Purchase', 'mean'),  
    Number_of_Transactions=('Purchase', 'count')  
) .reset_index()
```

```
customer_summary.head()
```

OUTPUT / RESULT

	User_ID	Total_Purchase	Average_Purchase	Number_of_Transactions
0	1000001	334093	9545.514286	35
1	1000002	810472	10525.610390	77
2	1000003	341635	11780.517241	29
3	1000004	206468	14747.714286	14
4	1000005	821001	7745.292453	106

INTERPRETATION

A customer-level summary is created with total spending, average transaction value and transaction count for each User_ID. The first customer has total Purchase 334,093 across 35 transactions, with an average of about 9,545.51.

Step 55: Top Customers Based on Average Purchase

CODE

```
customer_summary.sort_values(  
    by='Average_Purchase',  
    ascending=False  
) .head(10)
```

OUTPUT / RESULT

	User_ID	Total_Purchase	Average_Purchase	Number_of_Transactions
3801	1003902	1746322	18577.893617	94
4943	1005069	332823	18490.166667	18
5849	1005999	330227	18345.944444	18
1307	1001349	417743	18162.739130	23
3367	1003461	350174	17508.700000	20
98	1000101	1138265	17246.439394	66
2902	1002983	520509	16790.612903	31
5844	1005994	705057	16787.071429	42
576	1000592	234872	16776.571429	14
4361	1004474	282879	16639.941176	17

INTERPRETATION

Customers are sorted by average transaction value. User 1003902 has the highest displayed average, about 18,577.89, across 94 transactions.

Step 56: Create Age-wise Summary Table

CODE

```
age_summary = df.groupby('Age').agg(  
    Total_Purchase=('Purchase', 'sum'),  
    Average_Purchase=('Purchase', 'mean'),  
    Transactions=('Purchase', 'count')  
)
```

age_summary

OUTPUT / RESULT

	Age	Total_Purchase	Average_Purchase	Transactions
0	0-17	134913183	8933.464640	15102
1	18-25	913848675	9169.663606	99660
2	26-35	2031770578	9252.690633	219587
3	36-45	1026569884	9331.350695	110013
4	46-50	420843403	9208.625697	45701
5	51-55	367099644	9534.808031	38501
6	55+	200767375	9336.280459	21504

INTERPRETATION

An age-level summary combines total Purchase, average Purchase and transaction count. The 26-35 group has the largest total and transaction count, while 51-55 has the highest average Purchase.

Step 57: Create City-wise Summary

CODE

```
city_summary = df.groupby('City_Category').agg(  
    Total_Purchase=('Purchase', 'sum'),  
    Average_Purchase=('Purchase', 'mean'),  
    Transactions=('Purchase', 'count')  
)
```

city_summary

OUTPUT / RESULT

	City_Category	Total_Purchase	Average_Purchase	Transactions
0	A	1316471661	8911.939216	147720
1	B	2115533605	9151.300563	231173
2	C	1663807476	9719.920993	171175

INTERPRETATION

A city-level summary combines total Purchase, average Purchase and transaction count. City B has the largest total and transaction count, while city C has the highest average Purchase.

Step 58: Create Gender-wise Summary

```
CODE
gender_summary = df.groupby('Gender').agg(
    Total_Purchase=('Purchase', 'sum'),
    Average_Purchase=('Purchase', 'mean'),
    Transactions=('Purchase', 'count')
).reset_index()

gender_summary
```

OUTPUT / RESULT

	Gender	Total_Purchase	Average_Purchase	Transactions
0	F	1186232642	8734.565765	135809
1	M	3909580100	9437.526040	414259

INTERPRETATION

A gender-level summary shows total Purchase, average Purchase and transaction count. Male transactions have higher total and average Purchase and more transaction records in this dataset.

Step 59: Create Product Category Summary

```
CODE
product_summary = df.groupby('Product_ID').agg(
    Total_Purchase=('Purchase', 'sum'),
    Average_Purchase=('Purchase', 'mean'),
    Number_of_Transactions=('Purchase', 'count')
).reset_index()

product_summary.head()
```

OUTPUT / RESULT

	Product_ID	Total_Purchase	Average_Purchase	Number_of_Transactions
0	P00000142	12837476	11143.642361	1152
1	P00000242	3967496	10551.851064	376
2	P00000342	1296475	5313.422131	244
3	P00000442	441173	4795.358696	92
4	P00000542	807212	5417.530201	149

INTERPRETATION

A product-level summary combines total Purchase, average Purchase and transaction count for each product. The first rows illustrate the metrics for individual Product_ID values.

Step 60: Filter High Purchases

CODE

```
high_purchase = df[df['Purchase'] >= 10000]

high_purchase.head()
```

INTERPRETATION

The code filters transactions with Purchase >= 10,000 and displays the first matching records. The displayed rows are labeled High because they meet the Purchase_Level rule.

OUTPUT / RESULT						
	User_ID	Product_ID	Gender	Age	Occupation	City_Category \
1	1000001	P00248942	F	0-17	10	A
5	1000003	P00193542	M	26-35	15	A
6	1000004	P00184942	M	46-50	7	B
7	1000004	P00346142	M	46-50	7	B
8	1000004	P0097242	M	46-50	7	B
	Stay_In_Current_City_Years		Marital_Status		Product_Category_1	
\						
1	2		0		1	
5	3		0		1	
6	2		1		1	
7	2		1		1	
8	2		1		1	
	Product_Category_2		Product_Category_3		Purchase	Purchase_Level
1	6.0		14.0		15200	High
5	2.0		0.0		15227	High
6	8.0		17.0		19215	High
7	15.0		0.0		15854	High
8	16.0		0.0		15686	High

IV.

Visualizations

Step 61: 1. Customer Distribution by Age Group

OUTPUT / RESULT

CODE

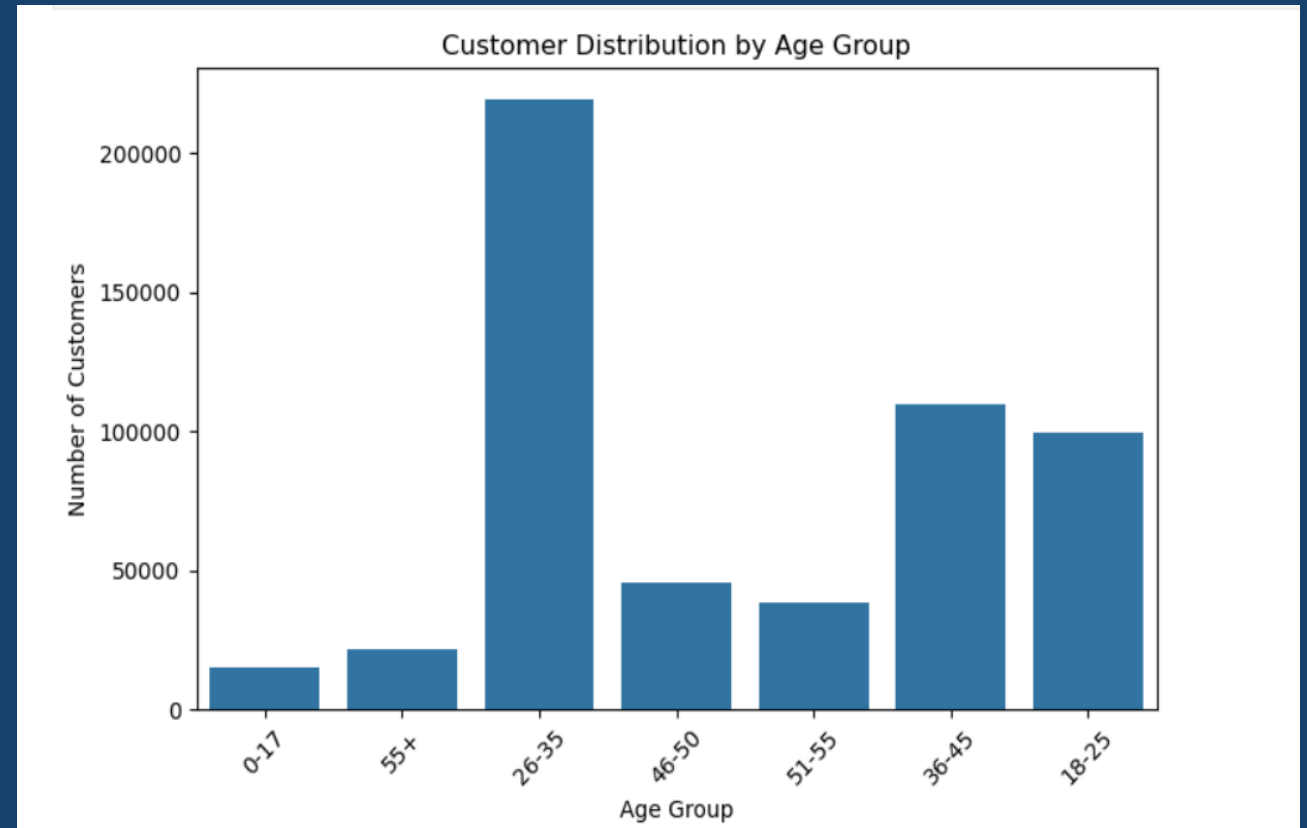
```
plt.figure(figsize=(8,5))

sns.countplot(data=df, x='Age')

plt.title("Customer Distribution by Age Group")
plt.xlabel("Age Group")
plt.ylabel("Number of Customers")

plt.xticks(rotation=45)

plt.show()
```



INTERPRETATION

The count plot visualizes the number of transaction records in each age group. The chart shows the age group 26-35 contribute the most, the age group 0-17 contribute the least records.

Step 62: 2. Gender Visualization

CODE

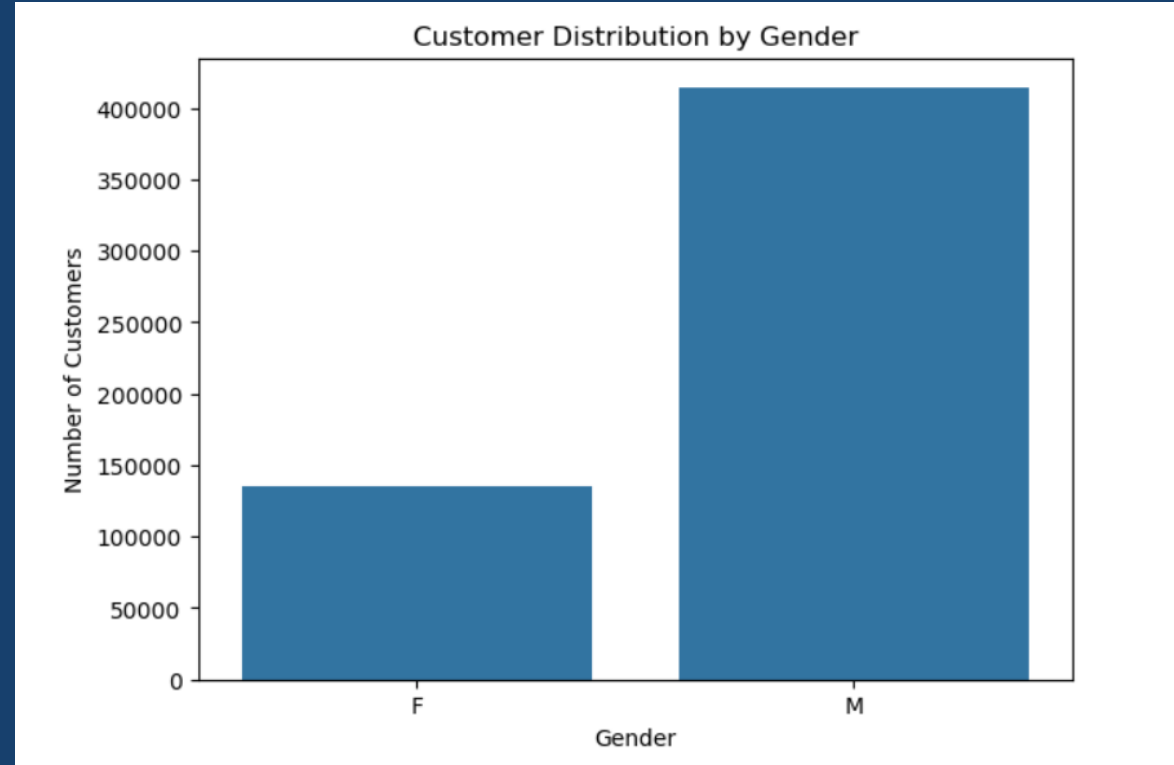
```
plt.figure(figsize=(7,5))

sns.countplot(data=df, x='Gender')

plt.title("Customer Distribution by Gender")
plt.xlabel("Gender")
plt.ylabel("Number of Customers")

plt.show()
```

OUTPUT / RESULT



INTERPRETATION

The count plot visualizes transaction-record counts by gender. It provides a direct visual comparison of male and female transaction frequencies.

Step 63: 3. Purchase Distribution

OUTPUT / RESULT

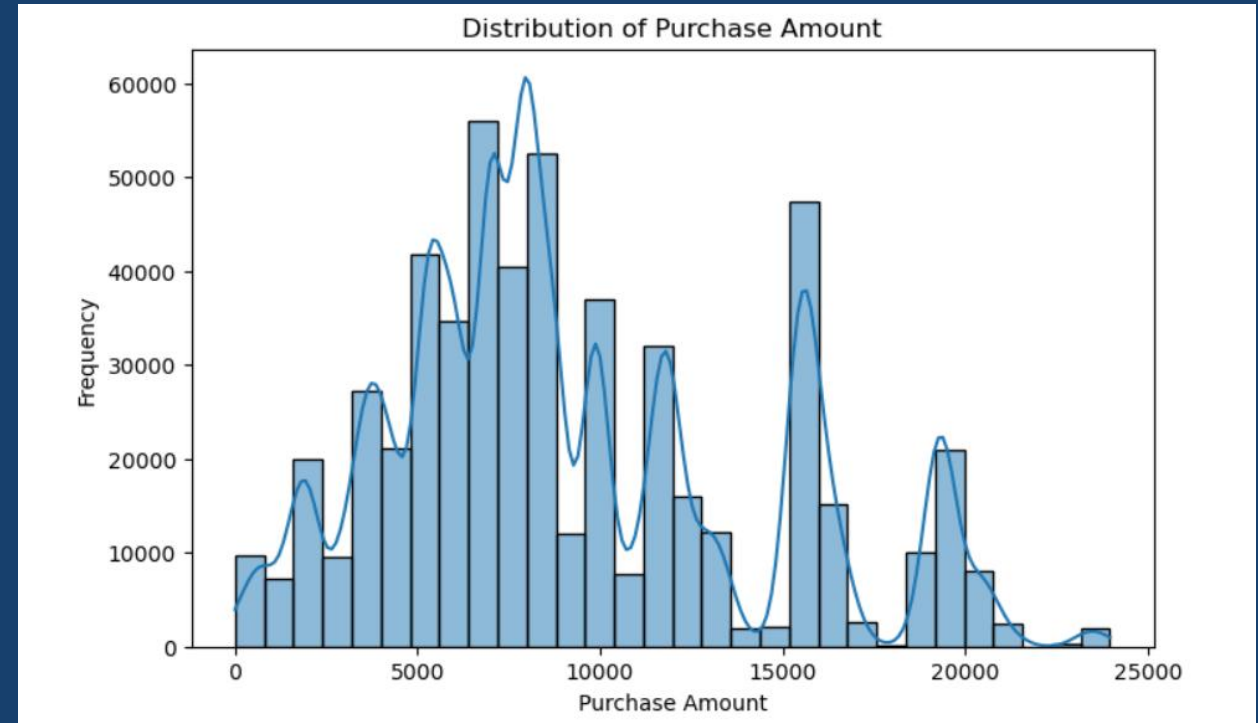
CODE

```
plt.figure(figsize=(8,5))

sns.histplot(df['Purchase'], bins=30, kde=True)

plt.title("Distribution of Purchase Amount")
plt.xlabel("Purchase Amount")
plt.ylabel("Frequency")

plt.show()
```



INTERPRETATION

The histogram shows how Purchase amounts are distributed across 30 bins, with a KDE curve to indicate the smoothed distribution curve. It helps inspect concentration and spread of transaction values.

- **Highest Frequency Peak :** The most prominent peak occurs around a purchase amount of 8,000 to 9,000, where the frequency approaches nearly 60,000 occurrences.
- **Sharp Drop-Off :** The frequency falls off drastically after 21,000.

Step 64: 4. Gender Sales

CODE

```
gender_sales =  
df.groupby('Gender')['Purchase'].sum()
```

```
gender_sales
```

OUTPUT / RESULT

```
Gender  
F      1186232642  
M      3909580100  
Name: Purchase, dtype: int64
```

INTERPRETATION

The grouped Series contains total Purchase by gender: F = 1,186,232,642 and M = 3,909,580,100. These values are then used for the following bar chart.

Step 65: 4. Gender Sales Visualization (Total Purchase by Gender)

CODE

```
plt.figure(figsize=(7,5))  
  
gender_sales.plot(kind='bar')  
  
plt.title("Total Purchase  
by Gender")  
plt.xlabel("Gender")  
plt.ylabel("Total  
Purchase")  
  
plt.xticks(rotation=0)  
  
plt.show()
```

OUTPUT / RESULT



groupby() groups the dataset according to gender.
['Purchase'].sum() calculates total purchases for each gender.

INTERPRETATION

The bar chart visualizes total Purchase by gender. The underlying totals show a much larger total for M than F in this dataset.

Step 66: 5. City Sales Chart

CODE

```
plt.figure(figsize=(7,5))

city_sales.plot(kind='bar')

plt.title("Total Purchase by City Category")
plt.xlabel("City Category")
plt.ylabel("Total Purchase")

plt.xticks(rotation=0)

plt.show()
```

OUTPUT / RESULT



INTERPRETATION

The bar chart visualizes total Purchase by city category. The underlying totals are B = 2,115,533,605, C = 1,663,807,476 and A = 1,316,471,661.

Step 67: 6. Product Category Chart

CODE

```
plt.figure(figsize=(10,6))

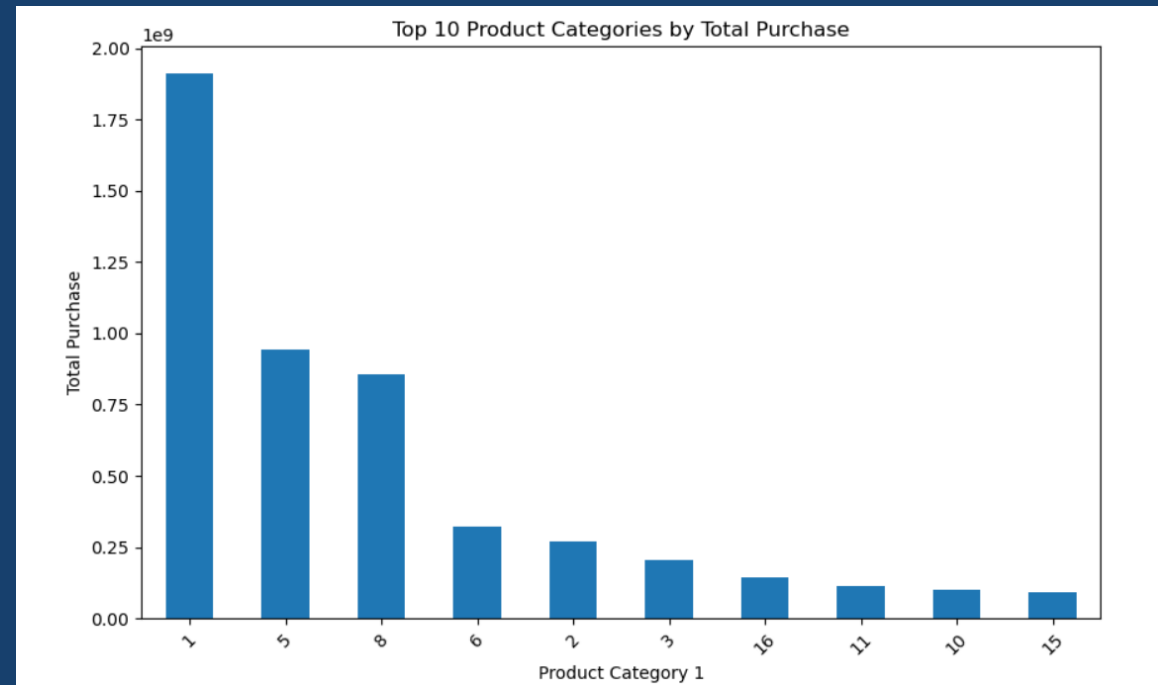
category1_sales.head(10).plot(kind='bar')

plt.title("Top 10 Product Categories by Total Purchase")
plt.xlabel("Product Category 1")
plt.ylabel("Total Purchase")

plt.xticks(rotation=45)

plt.show()
```

OUTPUT / RESULT



INTERPRETATION

The bar chart displays the top 10 Product_Category_1 values by total Purchase. The underlying ranking places category 1 first, followed by 5 and 8.

Step 68: 7. Product Category Count

OUTPUT / RESULT

CODE

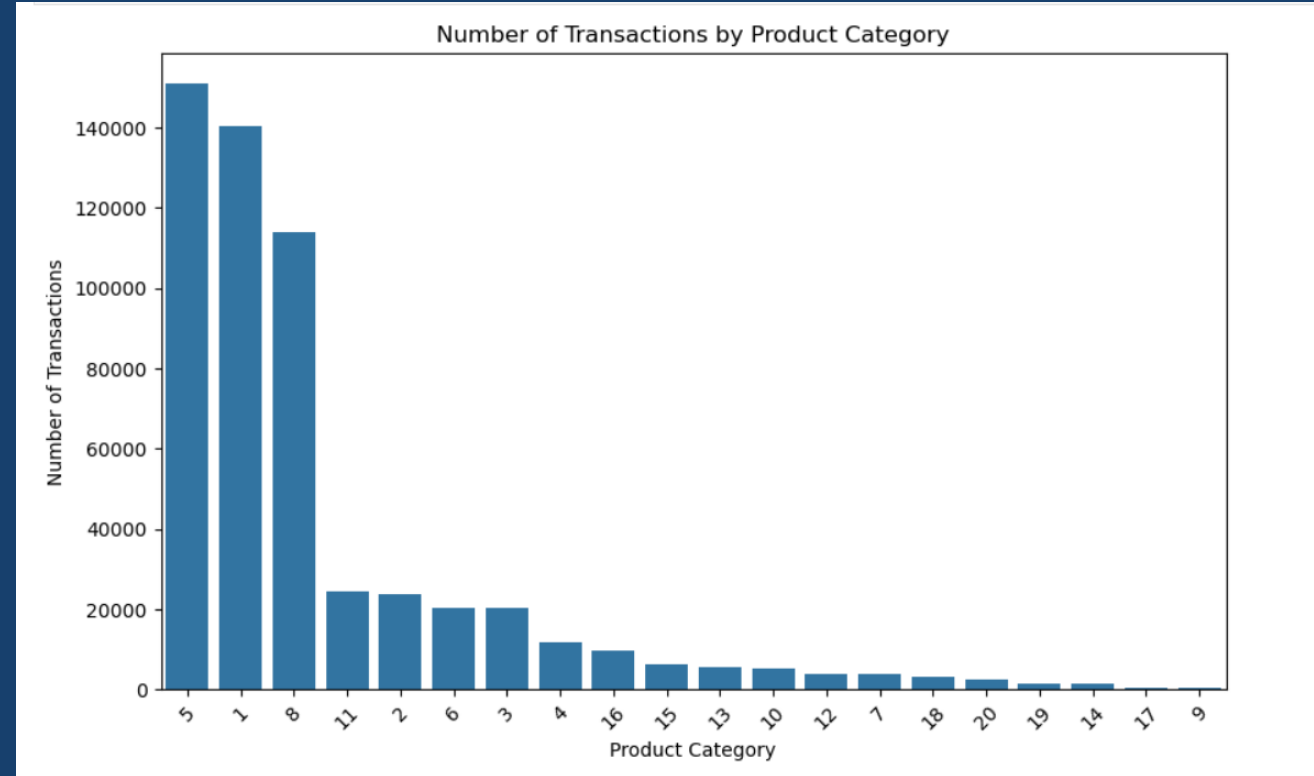
```
plt.figure(figsize=(10,6))

sns.countplot(
    data=df,
    x='Product_Category_1',
    order=df['Product_Category_1'].value_counts().index
)

plt.title("Number of Transactions by Product Category")
plt.xlabel("Product Category")
plt.ylabel("Number of Transactions")

plt.xticks(rotation=45)

plt.show()
```



INTERPRETATION

The count plot displays transaction frequency for Product_Category_1 and orders the bars from most frequent to least frequent. It is intended to identify categories with the most transaction records.

Step 69: 8. Purchase Box Plot

OUTPUT / RESULT

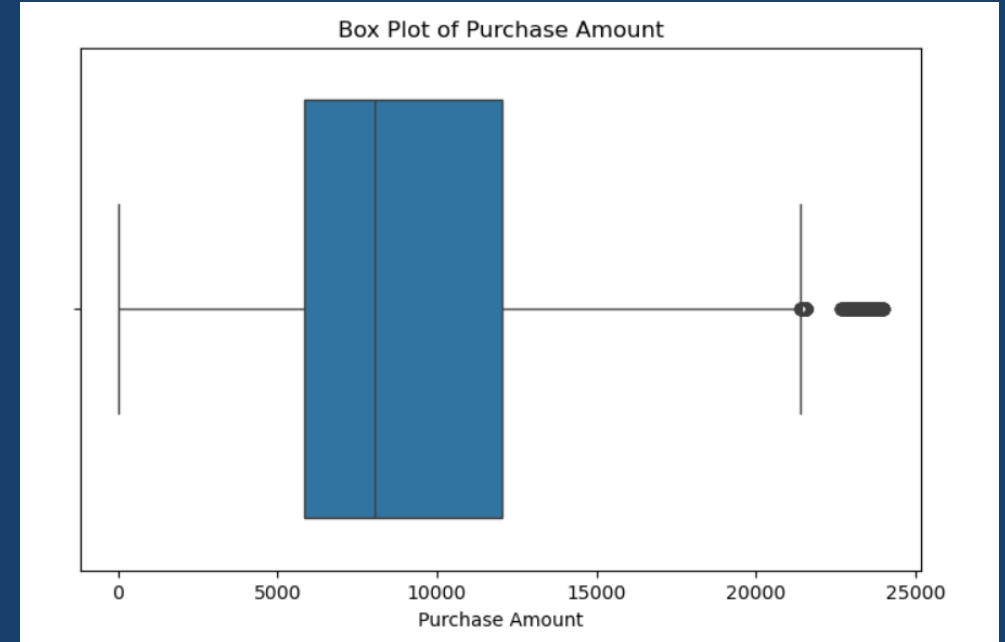
CODE

```
plt.figure(figsize=(8,5))

sns.boxplot(x=df['Purchase'])

plt.title("Box Plot of Purchase Amount")
plt.xlabel("Purchase Amount")

plt.show()
```



INTERPRETATION

The box plot summarizes the distribution of Purchase amounts using the median, quartiles and possible outliers. It helps assess median spread and unusually low or high transactions.

- Typical purchases are moderately low : between 5000 and 13,000.
- The data is symmetrically distributed. (extend out to roughly 0 to 21,000 equally). This indicates a very balanced, symmetrical distribution of standard purchase amounts.
- High-value outliers are present. There is an isolated data point near 22,000, showing that while rare, some individual purchases significantly exceed the standard customer spending range.

Step 70: 9. Purchase Analysis by Age and Gender

CODE

```
plt.figure(figsize=(10,6))
```

```
sns.barplot(  
    data=gender_age_sales,  
    x='Age',  
    y='Purchase',  
    hue='Gender'  
)
```

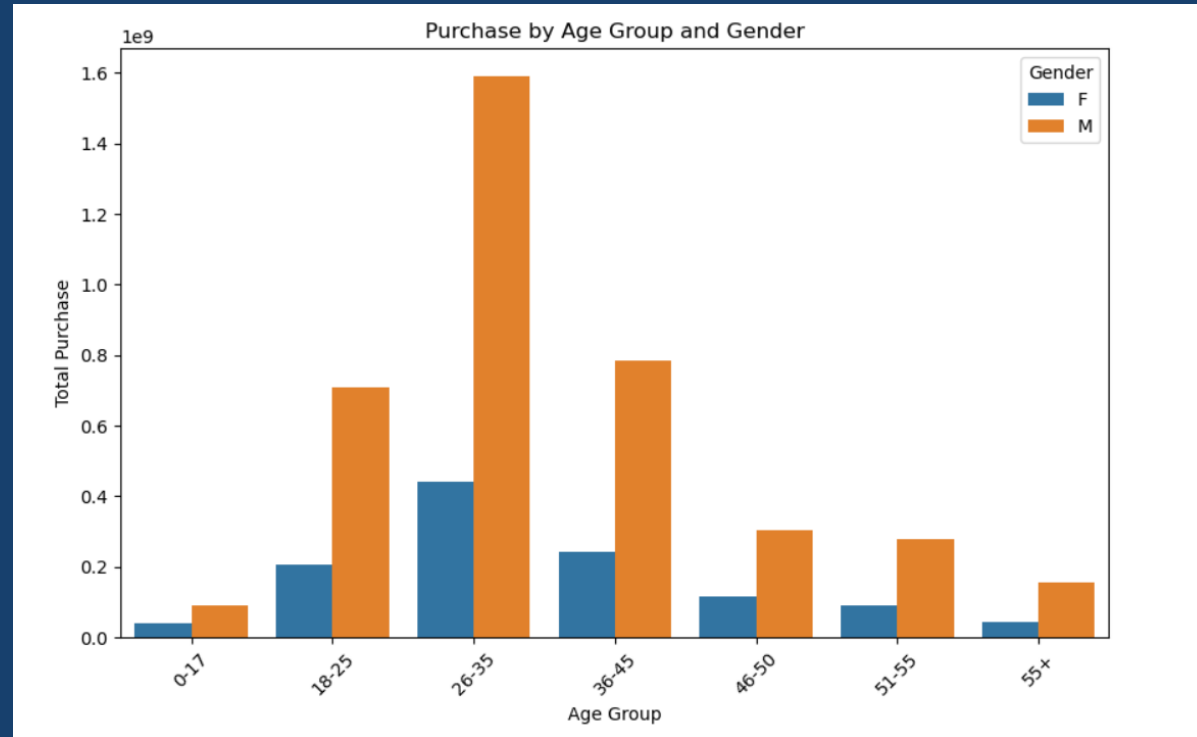
```
plt.title("Purchase by Age Group and Gender")  
plt.xlabel("Age Group")  
plt.ylabel("Total Purchase")
```

```
plt.xticks(rotation=45)
```

```
plt.show()
```

- `hue='Gender'` creates separate bars for genders.
- This allows us to compare purchasing patterns across age groups and gender

OUTPUT / RESULT



INTERPRETATION

The grouped bar chart compares total Purchase across age groups, separated by gender. The underlying table shows male totals above female totals in every age group shown.

Step 71: 10. City - gender Chart

OUTPUT / RESULT

CODE

```
plt.figure(figsize=(8,5))

sns.barplot(
    data=city_gender_sales,
    x='City_Category',
    y='Purchase',
    hue='Gender'
)

plt.title("Purchase by City Category and Gender")
plt.xlabel("City Category")
plt.ylabel("Total Purchase")

plt.show()
```



INTERPRETATION

The grouped bar chart compares total Purchase across city categories, separated by gender. The underlying values show male totals above female totals for A, B and C.

Step 72: 11. Correlation Analysis

CODE

```
numeric_columns =  
df.select_dtypes(include=np.number)
```

```
correlation = numeric_columns.corr()
```

```
correlation
```

Output / Result

	User_ID	Occupation	Marital_Status	Product_Category_1	Product_Category_2	Product_Category_3	Purchase
User_ID	1.000000	-0.023971	0.020443	0.003825	0.003896	0.003605	0.004716
Occupation	-0.023971	1.000000	0.024280	-0.007618	0.006712	0.012269	0.020833
Marital_Status	0.020443	0.024280	1.000000	0.019888	0.001099	-0.004629	-0.000463
Product_Category_1	0.003825	-0.007618	0.019888	1.000000	-0.067877	-0.385534	-0.343703
Product_Category_2	0.003896	0.006712	0.001099	-0.067877	1.000000	0.094750	0.052288
Product_Category_3	0.003605	0.012269	-0.004629	-0.385534	0.094750	1.000000	0.288501
Purchase	0.004716	0.020833	-0.000463	-0.343703	0.052288	0.288501	1.000000

INTERPRETATION

This repeats the numeric correlation calculation and stores the matrix in correlation. It prepares the data for the heatmap in the next step.

Step 73: 12. Correlation Heatmap

CODE

```
plt.figure(figsize=(12,8))

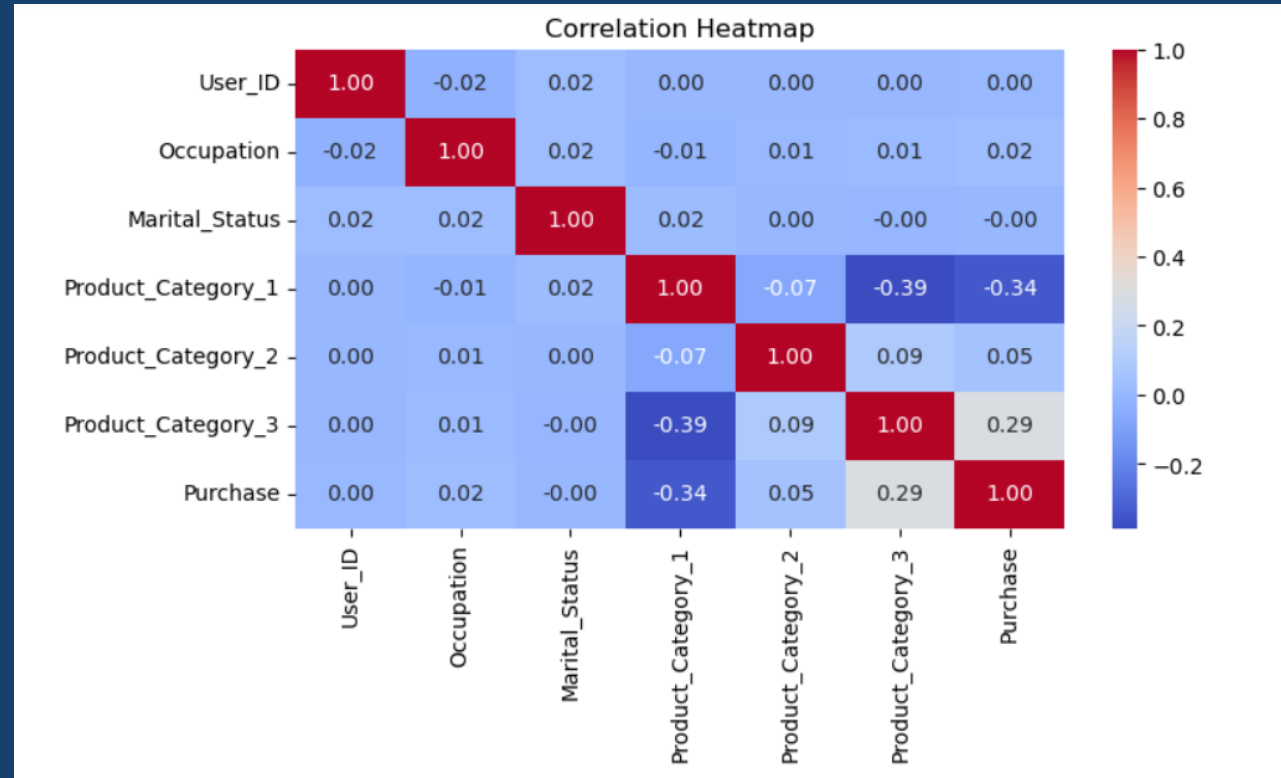
sns.heatmap(
    correlation,
    annot=True,
    cmap='coolwarm',
    fmt='.2f'
)

plt.title("Correlation Heatmap")

plt.show()
```

- 'annot=True' displays the correlation numbers.
- fmt='.2f' displays values to two decimal places.

OUTPUT / RESULT



INTERPRETATION

The heatmap displays the correlation matrix with numeric annotations. Values near +1 indicate strong positive linear association, values near -1 indicate strong negative association, and values near 0 indicate weak linear association.

Step 74: 13. Purchase vs Product Category (Box Plot)

OUTPUT / RESULT

CODE

```
plt.figure(figsize=(10,6))

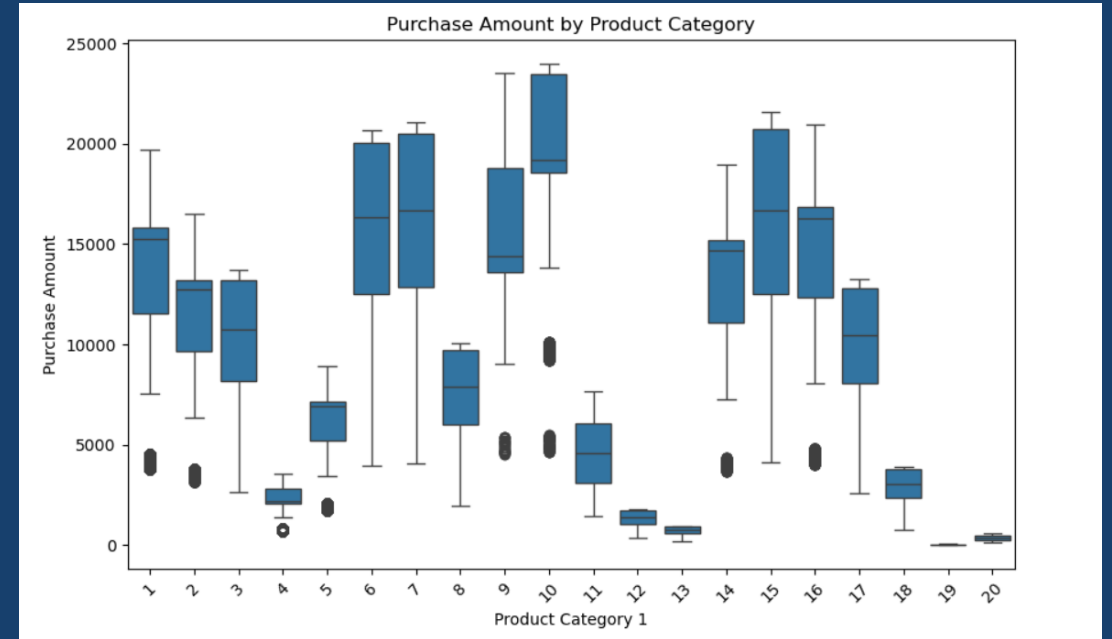
sns.boxplot(
    data=df,
    x='Product_Category_1',
    y='Purchase'
)

plt.title("Purchase Amount by
Product Category")
plt.xlabel("Product Category
1")
plt.ylabel("Purchase Amount")

plt.xticks(rotation=45)

plt.show()
```

The box plot compares Purchase distributions across Product_Category_1 categories. It helps examine median, spread and potential outliers for each category.



INTERPRETATION

The box plot compares Purchase distributions across Product_Category_1 categories. It helps examine median, spread and potential outliers for each category.

High value product categories: Category 10 has the highest median purchase amount and highest overall spending values, followed closely by Categories 6, 7, and 17.

Low value product categories: Categories 19, 20 show extremely low purchase amounts near zero with virtually no variation. Categories 4, 12 and 13 also consistently fall in the lower price / spending range (under 5000).

Data spread and variability: Categories with tall boxes and long whiskers (categories 6, 7, 9, 10, 15) show high variability, meaning customer spending in these categories ranges widely.

Categories with narrow boxes (categories 4, 12, 13, 19, 20) have very consistent purchase amounts.

Presence of Outliers: Several categories (1, 2, 4, 5, 9, 10, 14, 16) feature black dots below their lower whiskers. These indicate usually low individual transaction amounts compared to the bulk of purchases in those categories.

Step 75: 14. Top Customers Chart

OUTPUT / RESULT

CODE

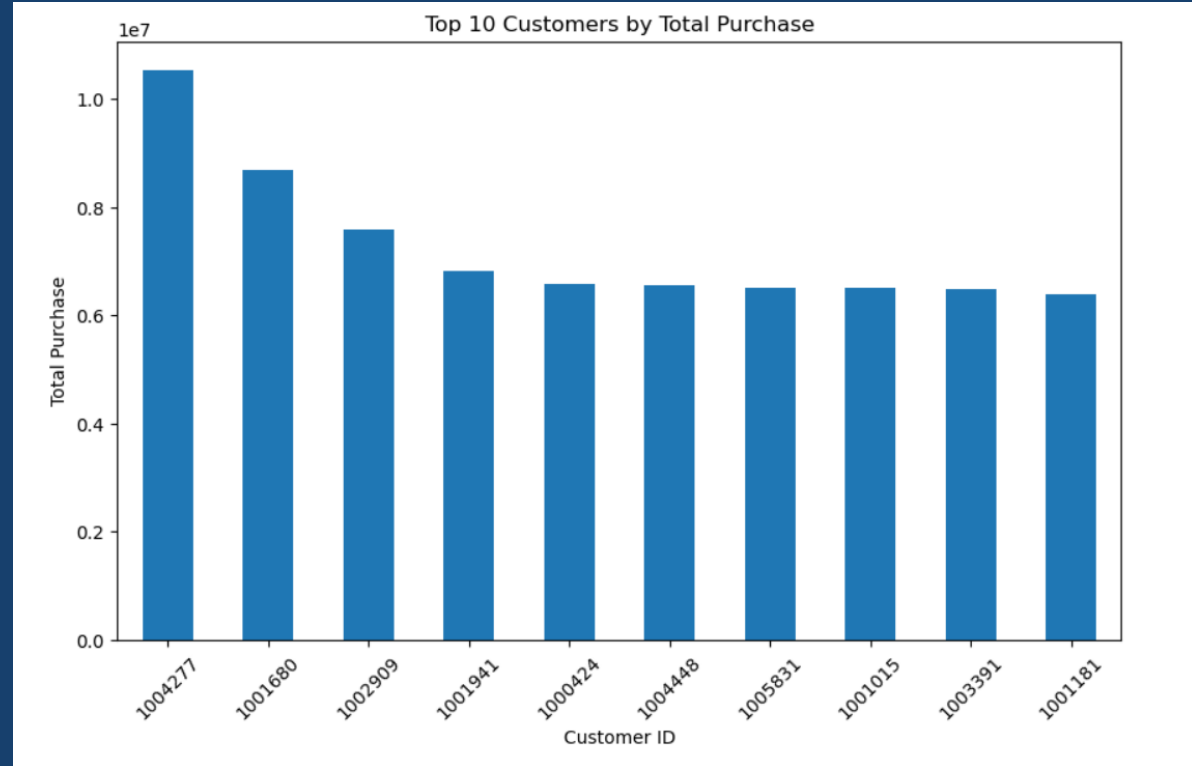
```
plt.figure(figsize=(10,6))

top_customers.plot(kind='bar')

plt.title("Top 10 Customers by Total Purchase")
plt.xlabel("Customer ID")
plt.ylabel("Total Purchase")

plt.xticks(rotation=45)

plt.show()
```



INTERPRETATION

The bar chart visualizes the ten customers with the highest total Purchase. The largest bar corresponds to customer 1004277 based on the underlying top_customers Series.

Step 76: 15. Top Products Chart

CODE

```
plt.figure(figsize=(10,6))

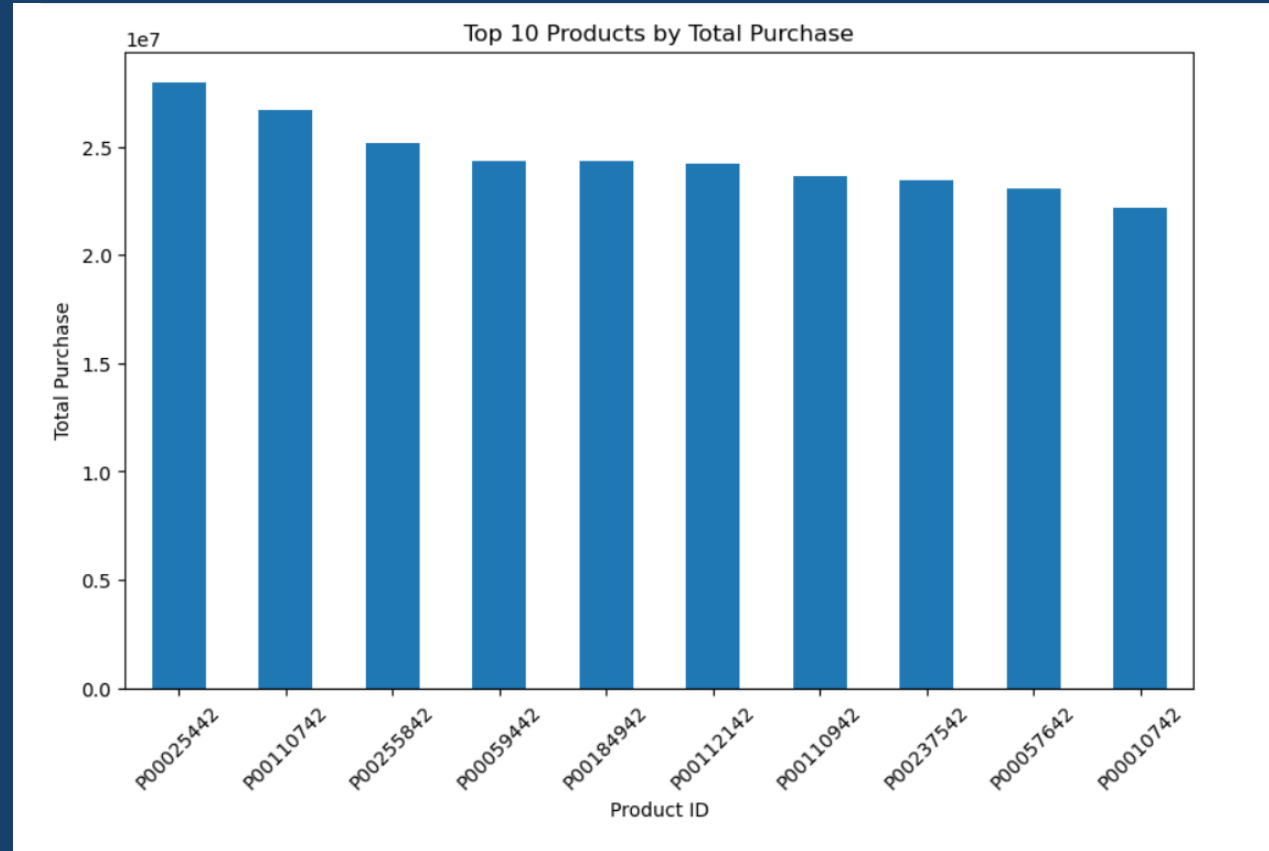
top_products.plot(kind='bar')

plt.title("Top 10 Products by Total Purchase")
plt.xlabel("Product ID")
plt.ylabel("Total Purchase")

plt.xticks(rotation=45)

plt.show()
```

OUTPUT / RESULT



INTERPRETATION

The bar chart visualizes the ten products with the highest total Purchase. The largest bar corresponds to product P00025442 based on the underlying top_products Series.

Step 77: 16. Average Purchase by Age Chart

OUTPUT / RESULT

CODE

```
plt.figure(figsize=(9,5))

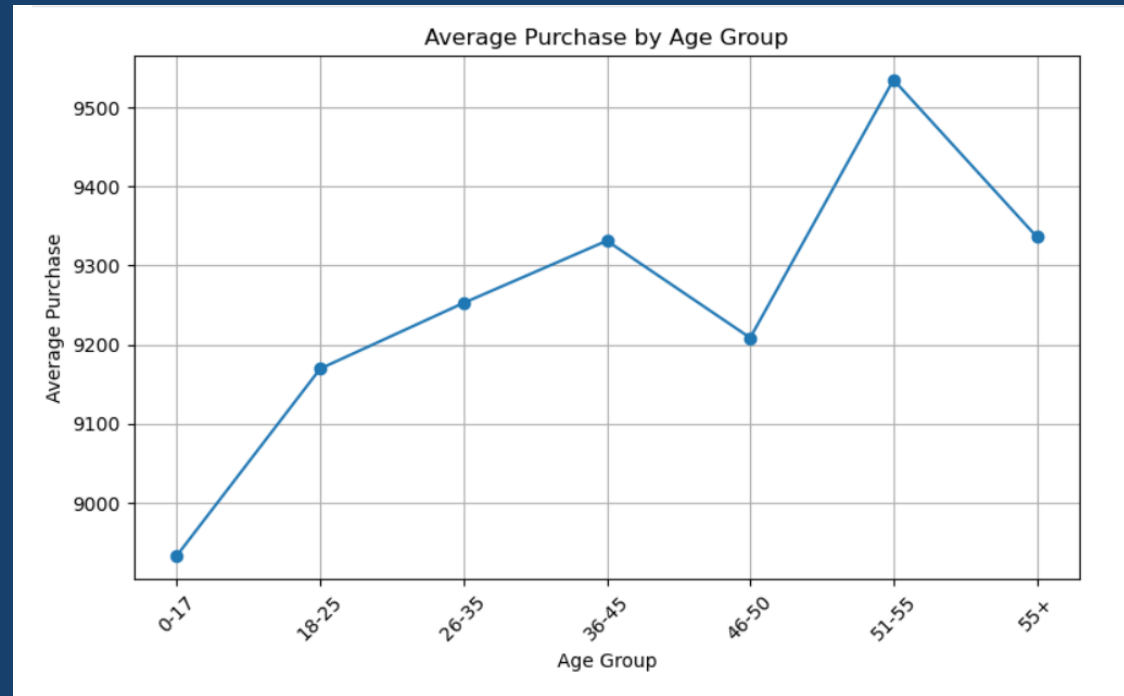
age_avg_purchase.plot(kind='line', marker='o')

plt.title("Average Purchase by Age Group")
plt.xlabel("Age Group")
plt.ylabel("Average Purchase")

plt.xticks(rotation=45)

plt.grid()

plt.show()
```



INTERPRETATION

The line chart shows average Purchase by age group. The underlying averages range from about 8,933.46 for 0-17 to about 9,534.81 for 51-55, allowing the pattern across age groups to be compared.

Step 78: 17. Marital Status Chart

OUTPUT / RESULT

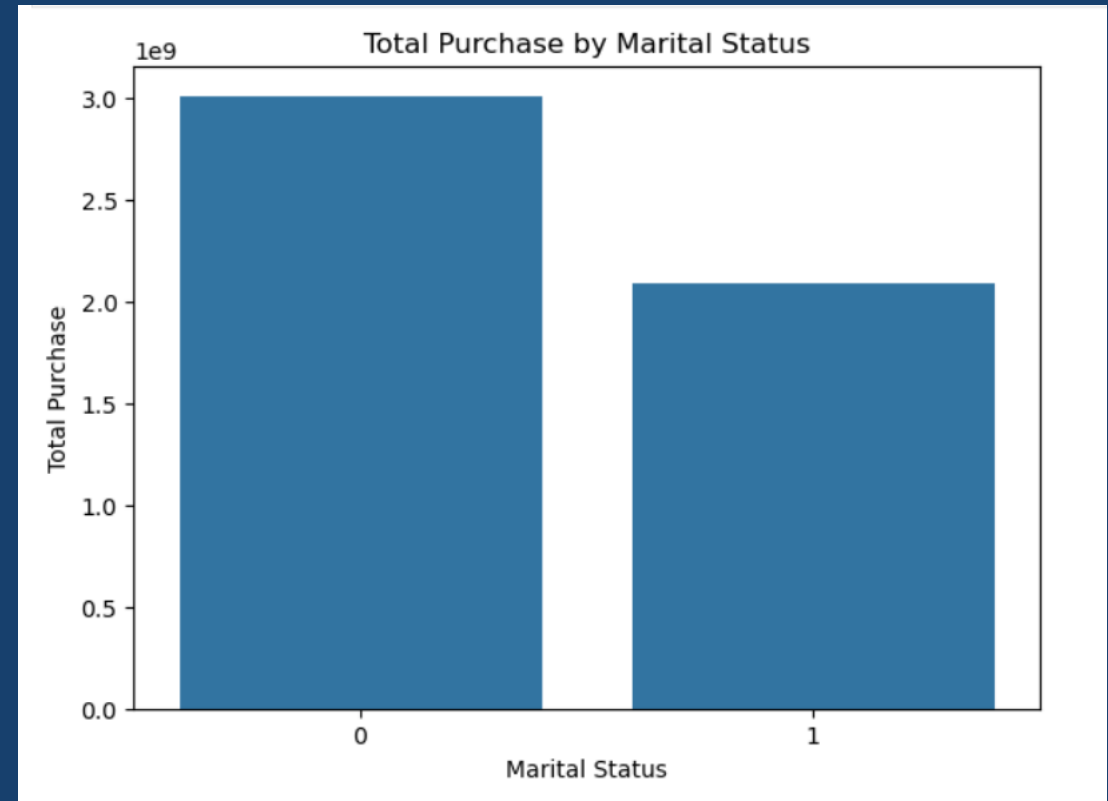
CODE

```
plt.figure(figsize=(7,5))

sns.barplot(
    x=marital_sales.index,
    y=marital_sales.values
)

plt.title("Total Purchase by Marital Status")
plt.xlabel("Marital Status")
plt.ylabel("Total Purchase")

plt.show()
```



INTERPRETATION

The bar chart compares total Purchase for the two Marital_Status codes. Status 0 has the larger total in the underlying data.

Step 79: 18. Stay Duration Chart

CODE

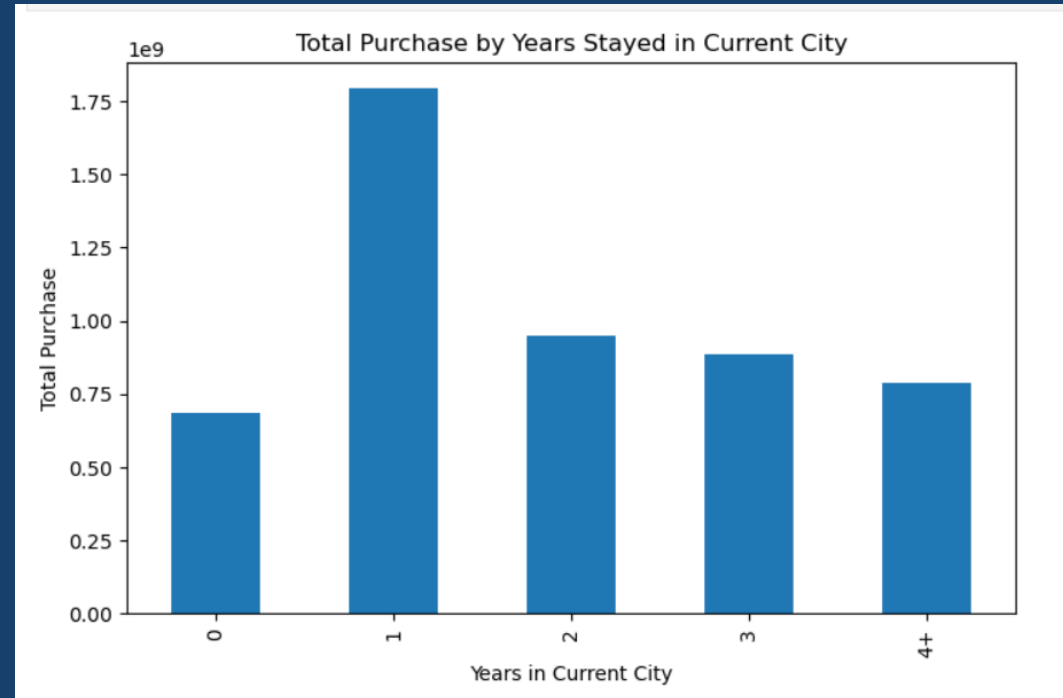
```
plt.figure(figsize=(8,5))

stay_sales.plot(kind='bar')

plt.title("Total Purchase by Years Stayed in Current City")
plt.xlabel("Years in Current City")
plt.ylabel("Total Purchase")

plt.show()
```

OUTPUT / RESULT



INTERPRETATION

The bar chart compares total Purchase by years stayed in the current city. The underlying totals are highest for 1 year and lowest for 0 years.

Step 80: 19. Pivot Table Visualization

CODE

```
plt.figure(figsize=(10,6))

sns.heatmap(
    pivot_table,
    annot=True,
    fmt='.0f'
)

plt.title("Purchase by Age and Gender")

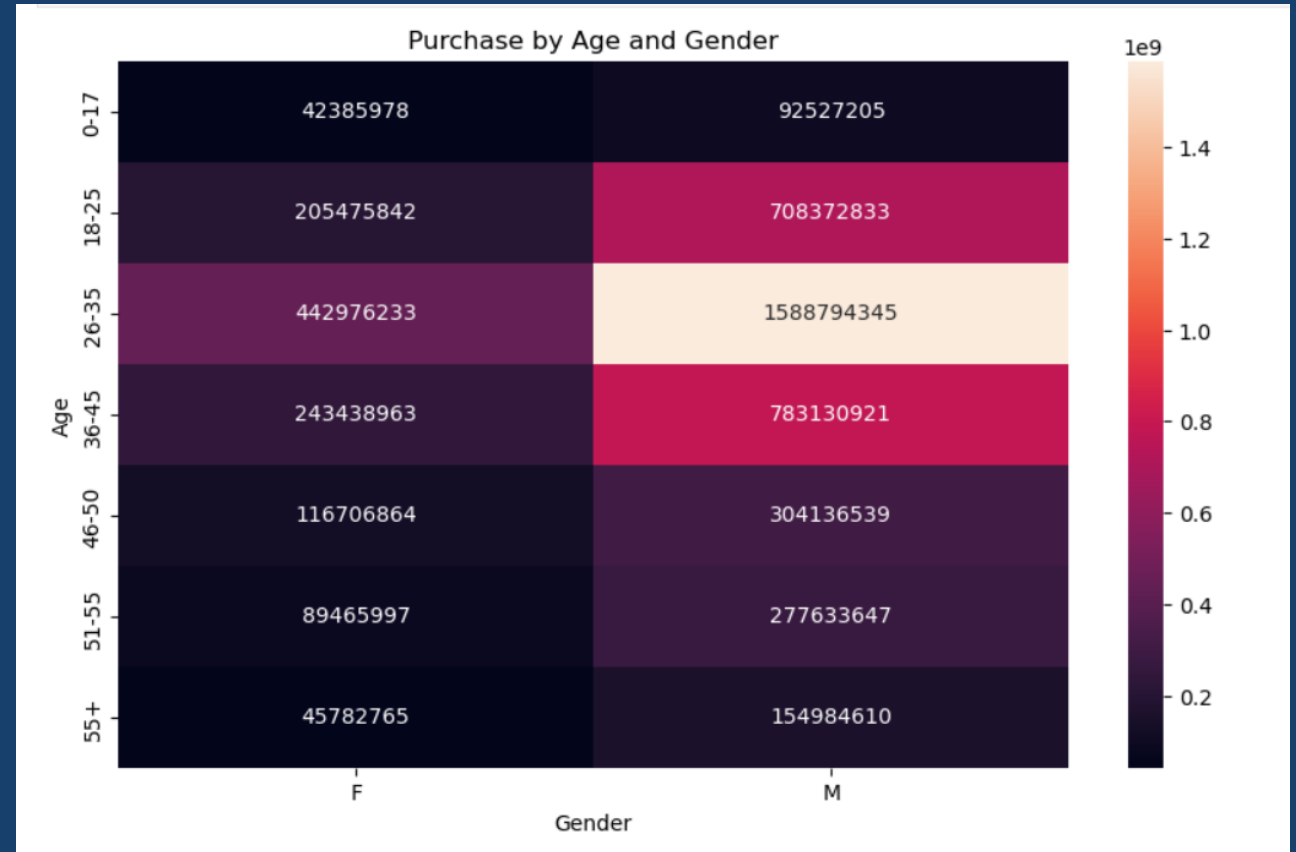
plt.show()
```

INTERPRETATION

The heatmap displays the Age × Gender total-Purchase pivot table with numeric annotations. It makes the combined demographic totals easier to compare.

Males under the age group 26-35 indicates the highest total purchase, while females under the age group 0-17 indicates the lowest.

OUTPUT / RESULT



Step 81: 20. Purchase Level Chart

OUTPUT / RESULT

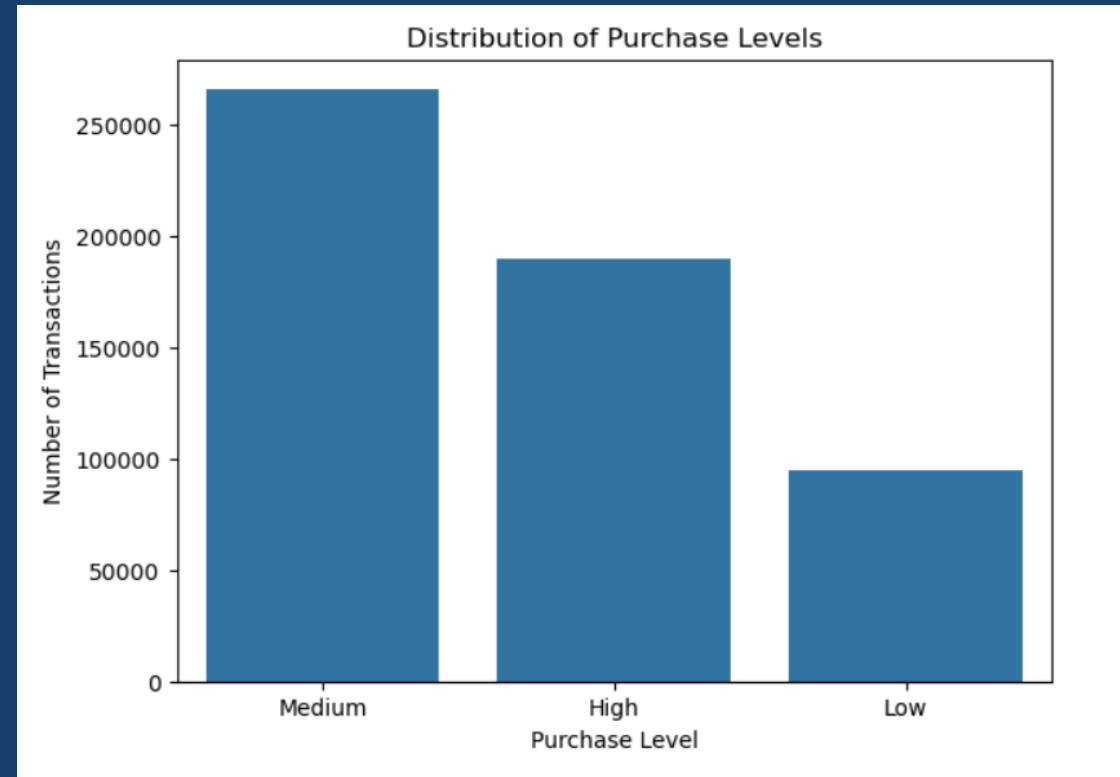
CODE

```
plt.figure(figsize=(7,5))

sns.countplot(
    data=df,
    x='Purchase_Level'
)

plt.title("Distribution of Purchase Levels")
plt.xlabel("Purchase Level")
plt.ylabel("Number of Transactions")

plt.show()
```



INTERPRETATION

The count plot shows the number of transactions in each Purchase_Level category: Low, Medium and High. The categories come from the thresholds defined earlier.

V

Insight Generation And Report

Step 82: Save Cleaned Dataset

CODE

```
df.to_csv(
    "Black-Friday-Cleaned-Dataset.csv",
    index=False
)
print("Cleaned dataset saved successfully")
```

OUTPUT / RESULT

Cleaned dataset saved successfully

INTERPRETATION

The cleaned DataFrame is exported to Black-Friday-Cleaned-Dataset.csv without the pandas index. The printed message confirms the save operation completed.

Step 83: Save Analysis Tables

CODE

```
age_summary.to_csv(
    "Age_Summary.csv",
    index=False
)

city_summary.to_csv(
    "City_Summary.csv",
    index=False
)

gender_summary.to_csv(
    "Gender_Summary.csv",
    index=False
)

product_summary.to_csv(
    "Product_Category_Summary.csv",
    index=False
)
```

OUTPUT

No printed/table output is stored for this cell. A successful run without an exception is the expected result.

INTERPRETATION

Four summary tables are exported as separate CSV files: Age_Summary.csv, City_Summary.csv, Gender_Summary.csv and Product_Category_Summary.csv. These files make the aggregated analysis reusable outside the notebook.

Step 84: Final Dataset Check

CODE

```
print("Final Dataset Shape:",  
df.shape)  
  
print("\nMissing Values:")  
print(df.isnull().sum())  
  
print("\nDuplicate Rows:")  
print(df.duplicated().sum())
```

OUTPUT/RESULT

```
Final Dataset Shape: (550068, 13)
```

```
Missing Values:
```

User_ID	0
Product_ID	0
Gender	0
Age	0
Occupation	0
City_Category	0
Stay_In_Current_City_Years	0
Marital_Status	0
Product_Category_1	0
Product_Category_2	0
Product_Category_3	0
Purchase	0
Purchase_Level	0
dtype: int64	

```
Duplicate Rows: 0
```

INTERPRETATION

The final dataset has 550,068 rows and 13 columns because Purchase_Level was added. All columns show 0 missing values and the duplicate-row count is 0, confirming the final cleaning checks passed.

Step 85: Final Statistical Summary

CODE

```
df.describe()
```

Output

	User_ID	Occupation	Marital_Status	Product_Category_1	Product_Category_2	Product_Category_3	Purchase
count	5.500680e+05	550068.000000	550068.000000	550068.000000	550068.000000	550068.000000	550068.000000
mean	1.003029e+06	8.076707	0.409653	5.404270	6.735436	3.841941	9263.968713
std	1.727592e+03	6.522660	0.491770	3.936211	6.215492	6.250712	5023.065394
min	1.000001e+06	0.000000	0.000000	1.000000	0.000000	0.000000	12.000000
25%	1.001516e+06	2.000000	0.000000	1.000000	0.000000	0.000000	5823.000000
50%	1.003077e+06	7.000000	0.000000	5.000000	5.000000	0.000000	8047.000000
75%	1.004478e+06	14.000000	1.000000	8.000000	14.000000	8.000000	12054.000000
max	1.006040e+06	20.000000	1.000000	20.000000	18.000000	18.000000	23961.000000

INTERPRETATION

The final descriptive statistics are shown after cleaning. Product_Category_2 and Product_Category_3 now include zeros, so their count is 550,068 and their means have changed from the original missing-data summary. Purchase statistics remain unchanged.

Key Insights

1. The analysis shows that customer demographics, purchasing behavior, product categories, city categories and sales patterns.
2. The age group 26-35 generates the highest total purchase.
3. The gender M generates the highest total purchase.
4. City category B has the highest total purchase.
5. Product category group1 generates the highest total purchase.
6. The analysis of customer-level purchases shows that the customer 1004277 has the highest total purchase among the customers in the dataset.
7. The correlation analysis indicates that most demographic variables have very weak or negligible relationships with purchase behavior.
In particular, User_ID ($r = 0.0047$), Occupation ($r = 0.0208$), and Marital_Status ($r = -0.0005$) show almost no correlation with Purchase, suggesting that these variables have little linear association with purchasing behavior.

Among the product categories, Product_Category_1 has a moderate negative correlation with Purchase ($r = -0.3437$), while Product_Category_3 has a moderate positive correlation ($r = 0.2885$).

Product_Category_2 shows a very weak positive correlation ($r = 0.0523$) with Purchase.

Overall, the results suggest that product category variables, particularly Product_Category_1 and Product_Category_3, have a stronger association with purchase behavior than the demographic variables.

However, correlation does not imply causation, and these results only describe linear relationships..

Key Insights(Continue...)

8. The final statistical summary indicates that the dataset contains 550,068 observations, with complete data across all variables. The Purchase variable has a mean of approximately 9,264 and a standard deviation of 5,023, indicating substantial variation in purchase amounts. Purchase values range from 12 to 23,961, with 50% of observations having purchase values below approximately 8,047.

Among the categorical variables, Occupation has a mean of 8.08, while Marital_Status has a mean of 0.41, indicating that the majority of observations belong to the non-married category if the coding is 0 = unmarried and 1 = married. Product categories also show considerable variation, particularly Product_Category_2 and Product_Category_3, as reflected by their relatively large standard deviations.

The major findings are:

1. The **26–35 age group** generated the highest total Purchase.
2. **Male customers** generated higher total Purchase than female customers.
3. **City Category B** had the highest total Purchase.
4. **Product Category 1** had the highest total Purchase among Product_Category_1 categories.
5. **Product Category 10** had the highest average Purchase among the displayed categories.
6. Customer and product analysis helped identify the highest-value customers and products.
7. The final dataset contained **no missing values and no duplicate rows**.

Final Conclusion

The Black Friday sales dataset was analyzed using Python and Pandas.

The project included data loading, data cleaning, missing-value handling, duplicate checking, statistical analysis, exploratory data analysis, groupby analysis, pivot tables, correlation analysis, and data visualization.

Original data: 550,068 rows × 12 columns.

After adding Purchase_Level: 550,068 rows × 13 columns.

Missing values in Product_Category_2 and Product_Category_3 were replaced with 0.

No exact duplicate rows were found.

Total Purchase: 5,095,812,742; average transaction: about 9,263.97.

Different visualizations were used to understand customer demographics, purchasing behavior, product categories, city categories, and sales patterns.

The analysis helped identify important patterns in customer purchasing behavior and product sales.

The findings can be used to understand customer preferences and support future sales and marketing decisions.

Cleaned and summary CSV files are exported for reuse.

THANK YOU

- Ms. Anusha Haridas (Faculty)
Madam, Thank you so much for your kind support and guidance for the completion of this project .
- Sona Raj (Co-Ordinator)
Madam, Thank you so much for your kind support and co-ordination.

THANK YOU